Proj Ideas (3/19)

(강병진)

약속 생성 + 초대장 + 지도 + 참여 투표 + 약속 알림

+ 더치페이 정산 + 송금

+ 사진 추억 다이어리(NRC 같은) + 친구 추가

(김시연)

1. 계절, 기온, 날씨에 따른 옷 추천 시스템

(서희정)

1. 청소년을 위한 재무구조 보고서 교육 프로그램: 보고서 내용을 쉽게 챗봇이 풀어주면서, 그래프 비교 시각화를 자동으로 만들어주는 주식 투자 교육 프로그램

DART: https://opendart.fss.or.kr/intro/infoApiList.do

https://comp.fnguide.com/SVO2/asp/SVD_Finance.asp?pGB=1&gicode=A003920&cID=&MenuYn=Y&ReportGB=B&NewMenuID=103&stkGb=701

(이설후)

1. 쇼핑몰 옷

2차 회의 (3/24)

4/14(월): 7주차 제안발표

청소년을 위한 재무 보고서 교육 프로그램

최종 서비스는 웹, 로컬로 돌린다고 생각하고 여유 되면 배포까지

<기술>

- **React 프론트엔드**
  - 사용자 입력, 학습 기능 제공
- **Spring Boot 백엔드**
  - 사용자 요청 처리
  - Python 서버에 분석 요청 전달 (POST 요청 등)
- **Python 서버 (Flask or FastAPI)**
  - OpenDART 데이터 수집, Pandas 분석
  - 시각화 이미지 or 수치 결과 반환

<Input>

재무보고서 API (XML/JSON)

https://opendart.fss.or.kr/intro/main.do

전자공시 OPENDART 시스템 | 공시정보 활용마당 | 재무정보 다운로드 | 재무정보 일괄다운로드

<거시적 기능>

1. openDART나 fnguide 웹스크래핑을 통해 재무제표 자료 끌어오기
2. 자료를 이용해 시각화
3. 관련 용어들, 그래프 해석 등을 ai를 이용해 설명
   a. 사용자 배경지식 수준 프롬프팅
   b. 이해 힘든 부분 드래그해서 넣으면 추가 설명 (꼬리질문으로 점점 더 쉽게)

<다음 안건>

가져온 데이터를 어떻게 가공해서 ai넣을건지 !

3차 회의 (3/26)

▼ (서희정) API 조사

https://www.fnspace.com/DataMart/MartList



유료 : (19년 기준) 학생은 월 5만원의 구독

<Open DART>

https://opendart.fss.or.kr/guide/detail.do?apiGrpCd=DS003&apiId=2022002

`fnlttSinglAcntAll` **(단일회사 전체 재무제표)**

→ 이 API는 *손익계산서, 재무상태표, 현금흐름표* 등을 **한꺼번에 묶어줌**

▼ (서희정) 역할 분담 아이디어

🧩 역할 분담 및 흐름 : Spring Boot + python + react(with chatgpt)

웹브라우저 없이도 Postman이나 curl로 테스트 가능한 백엔드 API + Python 기능을 먼저 구축!!

그럼 프론트엔드는 맨 마지막에 붙여도 OK

🔶 백엔드(Spring Boot)

- **기능 1: API 서버 구축**
  - 프론트엔드 또는 외부 요청에 대응해 데이터 반환
  - Python 팀에서 만든 분석/시각화 결과를 DB 또는 파일로 받아서 API화
  - 예: `/api/기업/{종목코드}` → 해당 기업 재무정보 반환
- **기능 2: 사용자 관리(선택)**
  - 로그인/북마크/학습 히스토리 등 부가 기능
- **기능 3: 프론트와 통신**
  - React에서 API 호출 → JSON 반환
  - AI 설명 결과를 Python에서 받아서 전달하는 중간다리 역할

🔶 Python 팀 (3명)

A. 데이터 수집 & 전처리 담당

- DART API 사용해서 재무제표 수집
- 기업별로 필수 항목(매출, 영업이익, 자산 등) 정리
- CSV 또는 DB로 백엔드에 전달

B. 시각화 & 분석 담당

- matplotlib / seaborn / plotly 등으로 그래프 생성 → 그래프 생성 자동화 set을 만들어야 할 듯
- 기업 간 비교 기능 구현
- 저장 형태는 PNG 또는 HTML로 만들어 전달 가능

  (혹은 plotly의 JSON 형태를 프론트에 넘겨서 그릴 수도 있음)

C. AI 설명 기능 담당

- OpenAI API 또는 HuggingFace 모델 이용
- 입력 텍스트(재무 용어/지표 등) → 초등/중학생 눈높이로 요약
- 꼬리질문 구조:

- 예: "ROE가 뭐야?" → "자기자본이익률이야~ 기업이 돈을 얼마나 잘 벌었는지 나타내!"
- 드래그된 부분에 대해 추가 질의 처리

▼ (서희정) 재무제표 개념 설명

1. 재무상태표 핵심:

- **부채비율 = 부채 / 자본 × 100**
  - 삼성전자: 92조 / 363조 ≈ 25.3%
  - 네이버: 15조 / 15조 = 100%
    👉 둘 다 건강한 편이지만, 삼성전자가 더 안정적

---

2. 손익계산서 핵심:

- **영업이익률 = 영업이익 / 매출**
  - 삼성전자: 36조 / 302조 ≈ 11.9%
  - 네이버: 1.2조 / 8조 ≈ 15%
    👉 네이버가 영업이익률은 더 높음 (= 팔아서 남기는 돈 비율이 높음)

---

3. 현금흐름표 핵심:

- **영업현금흐름이 '플러스'인지 & 순이익보다 많은지**
  - 이게 진짜로 돈 벌고 있는 기업인지 체크하는 핵심이야
  -

🎯 청소년 대상 펀더멘탈 분석:

이 3가지 지표만으로도 "좋은 기업 vs 위험한 기업" 비교 가능해

지표	의미	좋은 기업 기준
부채비율	빚이 자본보다 얼마나 많은지	100% 이하이면 안전
영업이익률	본업으로 얼마 남기는지	10% 이상이면 효율적
영업현금흐름 > 순이익?	진짜 돈 벌고 있나?	O면 건강한 구조

▼ (이설후) 전체적인 흐름 조사

프로젝트 개요

목표:

청소년이 기업의 재무보고서를 쉽게 이해할 수 있도록, DART 오픈API를 활용해 데이터를 불러오고

이를 **챗봇**이 쉽게 설명해주며, **시각적인 그래프**도 함께 제공하는 서비스 개발.

🔄 전체 프로젝트 흐름

1. **요구사항 정의 & 기술 스택 정하기**
2. **DART 오픈API 연동 → 기업 재무정보 수집**
3. **데이터 전처리 → 필요한 정보만 정리**
4. **자연어 설명 변환 → 쉽게 설명해주는 문장 생성**
5. **시각화 구현 → 차트로 재무정보 표현**
6. **챗봇 UI/UX 개발 → 대화 기반 설명 인터페이스**
7. **테스트 및 사용자 피드백 수렴**
8. **최종 배포 (웹 or 앱 형태)**

🧩 세부 계획

1. 주제 세분화 & 역할 분담

- **데이터 수집 파트:** DART 오픈API 사용, 기업코드 조회, 재무제표 불러오기
- **데이터 해석 파트:** 재무항목(매출, 자산, 부채 등) 분석 → 쉬운 문장으로 설명 생성
- **시각화 파트:** 그래프로 보여주기 (예: 수익 변화 추이, 자산 대비 부채 비율 등)
- **챗봇 파트:** 사용자의 질문에 대해 적절한 정보를 찾아 답변 생성 (Flask + Chat UI or React 기반)
- **기획/디자인 파트:** UX 흐름, 대상(청소년)에 맞는 언어톤 & 디자인

---

2. 데이터 수집 (OpenDART API)

- 필수 API
  - `corpCode.xml` : 기업 코드 조회

    - `fnlttSinglAcntAll.xml` : 재무제표 전체 항목 불러오기 (단일회사, 개별/연결)
- 필요한 항목
    - 매출액, 영업이익, 당기순이익, 자산총계, 부채총계, 자본총계 등

---

3. 데이터 전처리 & 변환

- API 응답 XML → pandas로 정리
- 전년 대비 증감률, 비율 계산 등 부가 정보 생성
- 키워드별로 분류: "성장 중", "적자 지속", "부채 과다" 등으로 태그화

---

4. 쉬운 설명 생성 (챗봇용)

- 예시:
    - “이 회사는 지난해보다 매출이 20% 늘었어요. 성장을 잘 하고 있네요!”
    - “이익은 줄었지만 여전히 흑자를 유지하고 있어요.”
- 템플릿 방식 또는 간단한 규칙기반 문장 생성기 활용

---

5. 데이터 시각화

- 라이브러리: `matplotlib`, `plotly`, `chart.js`, `recharts` (React 사용 시)
- 그래프 예시:
    - **막대 그래프**: 연도별 매출, 이익
    - **원형 그래프**: 자산 대비 부채 비율
    - **라인 차트**: 순이익 추이

---

6. 챗봇 개발

- 프론트: React 또는 HTML + JS (간단한 UI)
- 백엔드: Flask (Python 기반), DART API 연동 + 설명 생성 모듈 연결
- 챗봇 흐름 예시:
    1. 사용자: “삼성전자 재무보고서 보여줘”
    2. 챗봇: “2023년 매출은 279조 원, 이익은 34조 원이었어요. 그래프와 함께 볼까요?”

3. 그래프와 함께 설명 제공

---

7. 프로젝트 산출물

- 결과물 시연용 웹사이트 or 앱
- 기능 소개 PPT
- API 사용 예시, 모델 설명 문서화
- 오픈소스 저장소 (GitHub)

🛠️ 기술 스택 제안

파트	기술
백엔드	Python, Flask
프론트엔드	React or HTML/CSS/JS
데이터 시각화	matplotlib, plotly, chart.js
API	OpenDART 오픈API
데이터 처리	pandas
협업/배포	GitHub, Vercel, Netlify 등

<어떤 API Input?>

전자공시 OPENDART 시스템 | 개발가이드 | 상세

openDART “**단일회사 주요 재무지표**” JSON

<4/02(수) 한 페이지 발표>

주제 : 청소년을 위한 기업정보 분석을 통한 실생활 주식투자 교육 프로그램!

목표 : 청소년들의 주식 투자에 대한 부정적인 인식(투기) 타개, 건전한 투자법과 시민성 배양

필요성 :

최종 아웃풋: “금융 용어 설명 + 링크 연결” + “재무제표 분석 + 브리핑 + 챗봇”

1. 기본적인 교육 (재무제표 용어) 튜토리얼 + 링크 연결
    a. 가령 “ROE” 설명에서 "순이익" 클릭 → "순이익" 정의로 이동
2. 모든 회사 리스트 제공 + 섹터 나눠서 회사 리스트업 + 검색 기능
3. 회사 선택하면 재무제표, 그래프, AI 생성 브리핑 제공
    a. 재무상태표(대차대조표), 손익계산서, 현금흐름표 3가지 중 핵심적인 지표만
    b. 투자에 좋은 지표인지? 나쁜 지표인지?
4. 특정 내용 드래그해서 넣으면 AI 챗봇으로 설명(꼬리질문으로 점점 더 쉽게)

역할, 참고문헌, 특허/논문

▼ 주요지표 분석 Python 코드

```
def extract_core_metrics(df):
    result = {}

    # 재무제표별 필터링
    bs_df = df[df['sj_nm'] == '재무상태표']
    is_df = df[df['sj_nm'] == '손익계산서']
    cf_df = df[df['sj_nm'].str.contains('현금흐름표')]

    def get_amount(df, account):
        row = df[df['account_nm'] == account]
        if not row.empty:
            val = row.iloc[0]['thstrm_amount']
            return int(val.replace(',', ''))
        return None

    # 1. 부채비율
    부채 = get_amount(bs_df, '부채총계')
    자본 = get_amount(bs_df, '자본총계')
    if 부채 is not None and 자본:
        result['부채비율(%)'] = round((부채 / 자본) * 100, 2)

    # 2. 영업이익률
    영업이익 = get_amount(is_df, '영업이익')
    매출 = get_amount(is_df, '매출액')
```

```
    if 영업이익 and 매출:
        result['영업이익률(%)'] = round((영업이익 / 매출) * 100, 2)

    # 3. 현금흐름 체크
    영업현금 = get_amount(cf_df, '영업활동현금흐름') or get_amount(cf_df, '영업활동으로 인한 현금흐름')
    순이익 = get_amount(is_df, '당기순이익(손실)') or get_amount(is_df, '지배기업의 소유주에게 귀속되는 당기순이익(손실)')
    if 영업현금 is not None and 순이익 is not None:
        result['영업현금흐름 > 순이익'] = 영업현금 > 순이익
        result['영업현금흐름'] = 영업현금
        result['당기순이익'] = 순이익

    return result


company_dfs = {
    "삼성전자": final_df[final_df['corp_name'] == "삼성전자"]
    # 필요한 다른 기업도 추가 가능
}

for name, df in company_dfs.items():
    print(f"\n📊 {name} 핵심 재무 지표")
    metrics = extract_core_metrics(df)
    for k, v in metrics.items():
        print(f"{k}: {v}")
```

```
📊 삼성전자 핵심 재무 지표
부채비율(%): 25.36
영업현금흐름 > 순이익: True
영업현금흐름: 44137427000000
당기순이익: 15487100000000
```

교수님 피드백

제목 → 키워드에 주식, 청소년, 가치투자 조합해서 만들기

필요성 → 건강한 투자를 위해

목표 →

모의투자기능 했을때 결과가 어떻게 나오는지 분석이 가능한지 ?

다음주 수요일 발표 word한장 으로 → 핵심만 담아서

수행계획서 목차를 다 넣은 후에 내용 채울 수 있는것만 채우고 못 채우는건 추후

→ 일정은 교수님께서 주신 일정

역할 분담

참고문헌은 별도 가능

성과는 자유롭게

open DART

API key : 8fca130870210daea83ce8e8060e4edb6e1b5e0a

▼ api 키 예시

```
import requests
import pandas as pd
import time

api_key = '  8fca130870210daea83ce8e8060e4edb6e1b5e0a'

companies = {
"삼성전자": "00126380",
"SK하이닉스": "00164742"
}

bsns_year = '2023'
reprt_code = '11011'  # 사업보고서
fs_div = 'CFS'        # 연결재무제표

def get_financial_statement(corp_name, corp_code):
url = 'https://opendart.fss.or.kr/api/fnlttSinglAcntAll.json'
params = {
        'crtfc_key': api_key,
        'corp_code': corp_code,
        'bsns_year': bsns_year,
        'reprt_code': reprt_code,
```

```
    'bsns_year': bsns_year,
    'reprt_code': reprt_code,
    'fs_div': fs_div
}
res = requests.get(url, params=params)
data = res.json()

if data['status'] == '000':
    df = pd.DataFrame(data['list'])
    df['corp_name'] = corp_name
    return df[['corp_name', 'sj_nm', 'account_nm', 'thstrm_amount']]
else:
    print(f"{corp_name}: {data['message']}")
    return None
```

4차 회의 (3/31)

▼ (강병진) 모든 회사 리스트 제공 + 섹터 나눠서 회사 리스트업 + 검색 기능

1. 기능, 특징

- **기능**
  - 전체 기업 리스트 조회
  - 섹터별 필터 기능
  - 키워드 기반 기업 검색
- **특징**
  - RESTful API 기반의 백엔드 서버(Spring Boot)
  - JSON 포맷으로 클라이언트에 데이터 제공
  - 빠른 검색을 위한 DB 인덱싱 적용

---

2. 최종 설계 결과물

- **형태**: 백엔드 API 서버 (Spring Boot 기반 소프트웨어 프로토타입)
- **외적 형태**: 프론트엔드에서 호출 가능한 기업 정보 API 제공

---

3. 시스템 구성과 기능

🔷 대표 기능: 회사 리스트 제공

- 하위 기능:
  - `/api/companies` : 전체 조회
  - `/api/companies/search?keyword=삼성` : 키워드 포함 기업 검색
  - `/api/companies/sector?sector=IT` : 섹터 필터
- 달성 수준:
  - 필터링 및 검색 정확도 100%
  - 빠른 API 응답 속도

---

4. 다이어그램

✅ 유스케이스 다이어그램

- 사용자는 "기업 전체 보기", "섹터별 보기", "검색" 3가지 기능 사용
- 시스템은 해당 요청에 따라 필터링된 데이터를 응답

✅ 시스템 블록다이어그램

```
[사용자 요청]
    ↓
[프론트엔드 화면 (검색 입력, 버튼 클릭)]
    ↓ HTTP 요청
[Spring Boot API 서버]
    ↓ JPA Repository 호출
[MySQL DB]
    ↓
[검색 결과 반환 → 프론트]
```

✅ 시퀀스 다이어그램 (검색 예시)

```
사용자 → 프론트엔드: "삼성" 검색
프론트 → API 서버: GET /api/companies/search?keyword=삼성
API 서버 → DB: SELECT * FROM company WHERE name LIKE '%삼성%'
DB → API 서버: 검색 결과 반환
API 서버 → 프론트: JSON 응답 반환
프론트 → 사용자: 검색 결과 표시
```

✅ (3) 대안 도출 및 구현 계획

1. 대안 비교

대안	장점	단점
모든 데이터 프론트에서 필터	빠름, 서버 부담 적음	데이터 양 많으면 느림
서버에서 필터 및 검색 (선택안)	대용량 처리 유리	서버 성능 요구
Open API 활용	실시간 데이터 확보 가능	의존성 큼, 과금 가능성

→ **서버 측 필터/검색 처리 방식 선택** (검색 속도 및 대용량 확장성 확보)

---

2. 구현 계획

단계	내용
1	MySQL에 기업 리스트 데이터 입력
2	Spring Boot로 API 서버 구성 (JPA 기반)
3	전체 조회, 섹터 필터, 검색 기능 구현
4	프론트와 연결 및 테스트
5	성능 최적화 (인덱스, 캐시 등)

---

3. 자료 구조 / 알고리즘

- 자료구조: RDB 기반 Table (회사명, 섹터, 종목코드 등)
- 알고리즘:
  - 검색 시 `LIKE` 쿼리 + 인덱싱
  - 섹터는 ENUM 또는 TEXT 필터링

---

4. 플로우차트 (검색 흐름)

```
[사용자 검색 입력]
    ↓
[프론트엔드 GET 요청]
    ↓
[Spring Boot 컨트롤러]
    ↓
[서비스 → JPA 쿼리 실행]
    ↓
[DB 검색 결과 반환]
    ↓
[서비스 → 컨트롤러 → 프론트로 응답]
```

▼ (서희정) 기본적인 교육 (재무제표 용어) 튜토리얼 + 링크 연결

1. 소개 (Introduction)

- 삼성전자라는 실제 기업을 예시로 사용.
- 재무제표 분석의 목표: 삼성전자의 **재무 상태, 수익성, 현금 흐름**을 이해하기.

- 사용 도구: **Python, Open DART API, React (웹 인터페이스)**

📱 모바일 재무제표 분석 웹 앱 구조

🔍 기능 개요

1. **특정 기업 선택 페이지**
2. **탭 메뉴 (재무제표, 손익계산서, 현금흐름표, Valuation, 총평)**
3. **각 탭마다 해당 기업의 지표를 분석하여 텍스트와 이미지로 표현**
4. **하단에는 데이터베이스(DB), API 연결, 레이아웃 조정 기능 등 포함**

- React로 **탭 컴포넌트**를 구현하여 각 메뉴를 전환하며 확인 가능
- Flask로 **DART API에서 데이터 불러오기**
- 각 분석 결과를 **텍스트와 그래프**로 표현

📝 React + Flask (Python)로 구현하기

- **Frontend (React):** 사용자가 볼 수 있는 화면을 구현
- **Backend (Flask):** Open DART API로 데이터를 불러오고 분석하는 기능 제공
- **Database (MySQL):** 사용자 데이터와 분석 결과를 저장

📱 React 구성 요소

1. **메인 페이지 (기업 선택 페이지)**
2. **탭 메뉴 (Tabs Component)**
3. **분석 결과 출력 컴포넌트 (Text Analysis, Graph Visualization)**

구현 예시

```
// App.jsx
import React, { useState } from 'react';
import { Tabs, Tab } from "@/components/ui/tabs";
import AnalysisTab from './components/AnalysisTab';

const App = () ⇒ {
```

```
  const [activeTab, setActiveTab] = useState("재무제표");

  const tabs = ["재무제표", "손익계산서", "현금흐름표", "Valuation", "총평"];

  return (
    <div className="p-4">
      <h1 className="text-2xl mb-4">삼성전자 재무 분석</h1>
      <Tabs value={activeTab} onChange={setActiveTab}>
        {tabs.map((tab) => (
          <Tab key={tab} value={tab}>{tab}</Tab>
        ))}
      </Tabs>
      <AnalysisTab activeTab={activeTab} />
    </div>
  );
};

export default App;
```

AnalysisTab.jsx (각 탭별 내용 렌더링)

```
// AnalysisTab.jsx
import React from 'react';

const AnalysisTab = ({ activeTab }) => {
  const content = {
    "재무제표": "삼성전자의 최근 재무제표 데이터입니다. 자산, 부채, 자본을 분석합니
    "손익계산서": "삼성전자의 매출, 영업이익, 순이익을 분석합니다.",
    "현금흐름표": "삼성전자의 영업활동, 투자활동, 재무활동의 현금흐름을 분석합니다
    "Valuation": "삼성전자의 가치 평가. PER, PBR 등 주요 지표 분석.",
    "총평": "삼성전자의 종합 평가: 최근 매출이 상대적으로 줄었어요! (최근 데이터 참
  };

  return (
    <div className="mt-4 p-4 border rounded shadow-sm">
      <h2 className="text-xl mb-2">{activeTab}</h2>
```

```
      <p>{content[activeTab]}</p>
    </div>
  );
};

export default AnalysisTab;
```

Flask 백엔드 (Python) 예시

```
from flask import Flask, request, jsonify
import requests

app = Flask(__name__)

@app.route('/get_data', methods=['GET'])
def get_data():
    corp_code = request.args.get('corp_code')
    api_key = "YOUR_API_KEY"
    url = f"https://opendart.fss.or.kr/api/fnlttSinglAcnt.json?crtfc_key={api_k
    response = requests.get(url)
    data = response.json()
    return jsonify(data)

if __name__ == '__main__':
    app.run(debug=True)
```

▼ (서희정) 논의하고 싶은 부분

1. pc로 하면 fnguide랑 다를 바가 없는 것 같음. 모바일 웹으로 만들어보는 건 어떤가여? 요즘 주식도 핸드폰 mts로 하니까요!
2. 청소년을 위한 재무제표 교육 프로그램이었는데, 어째 조사를 하면 할수록 주식 투자를 위한 재무분석 DB를 만들게 되는 느낌입니다. 그래서 정체성 혼란이 오고 있어요ㅜ 청소년이라 해서 기본적인 것만 가르키자니 뭔가 이상해지는 것 같아요 차라리 그냥 교육프로그램으로 하고 난이도를 기초 용어에서 기본필수적인 건 다 알려주는 부분으로 하는 건 어떠신가용
3. 프로그램 이름 : 튜자 (투자에 1을 더해 튜자ㅋㅋㅋㅋㅋ)

▼ (서희정) 재무제표 분석 지표 추가 (by 경영학과 교수님 조언 : 아무리 그래도 PER, PBR은 봐야지)

1. 기업의 건강 상태 파악 (재무 안정성 분석) 지표

- **유동성 비율**: 단기 부채를 갚을 수 있는 능력을 측정.
  - **유동비율 (Current Ratio)** = 유동자산 / 유동부채 (100% 이상이 바람직)
  - **당좌비율 (Quick Ratio)** = (유동자산 - 재고자산) / 유동부채 (당좌비율이 높을수록 좋음)
- **부채비율 (Debt Ratio)**: 회사의 재정 구조 안정성을 평가.
  - **부채비율** = 총부채 / 총자본 (100% 이하가 안정적)
  - 업종마다 다르지만, 너무 높은 부채비율은 위험 신호
- **이자보상비율 (Interest Coverage Ratio)**: 영업이익으로 *이자 비용을 얼마나 감당*할 수 있는지 보여줌.
  - **이자보상비율** = 영업이익 / 이자비용 (200% 이상이면 안전)

---

2. 성장 가능성 평가 (성장&수익성 분석) 지표

- **매출 성장률 (Revenue Growth Rate)**: 매출이 얼마나 증가하고 있는지 평가.
  - **매출 성장률** = [(금년도 매출 - 전년도 매출) / 전년도 매출] × 100
  - 꾸준히 증가하는 것이 좋지만, 지나치게 빠르면 지속 가능성을 의심.

  ▼ 판단 기준

  - (급격한 성장의 배경이 **비정상적인 원인**일 수 있음. ex) 일회성 이벤트, 정부 보조금, 혹은 지나치게 공격적인 마케팅 등)
  - 📈 매출 성장률 판단의 **일반적인 기준**:
    - 연간 성장률이 **20~30% 이상**이면 빠른 성장으로 간주될 수 있어요.
    - 산업 평균 성장률보다 **2배 이상 높을 때**도 의심해볼 필요가 있음.
  - 📊 **경우에 따른 해석**:
    - **스타트업이나 신산업**: 초반에 빠르게 성장하는 게 자연스럽기 때문에 50% 이상의 성장률도 괜찮을 수 있어요.
    - **성숙한 산업**(예: 전통 제조업): 연 10% 이상의 성장은 비정상적으로 빠를 수 있음.

- **영업이익률 (Operating Profit Margin)**: 매출에서 영업비용을 제외한 영업이익의 비율.

- **영업이익률** = 영업이익 / 매출 × 100
- 산업 평균보다 높은 것이 좋음.

▼ **산업 평균 계산 방법**:

1. **동일 산업 내 주요 기업들의 영업이익률 수집**

   예를 들어, 반도체 산업이라면 삼성전자, SK하이닉스 등 주요 기업의 영업이익률을 모두 조사.

2. **평균 값 산출 (산술 평균)**산업 평균 영업이익률=기업 수개별 기업들의 영업이익률 총합

$$\text{산업 평균 영업이익률} = \frac{\text{개별 기업들의 영업이익률 총합}}{\text{기업 수}}$$

3. **중간값 사용 (Median)** - 극단적인 값(예: 아주 높은 이익률)을 제외하고 판단하고 싶을 때 사용.

- 🔍 **데이터 출처**:
  - Open DART API (국내 기업), Yahoo Finance, Reuters 등에서 산업별 주요 기업의 데이터를 수집하여 계산.

- **ROE (Return on Equity, 자기자본이익률)**: 주주의 투자 대비 수익률을 측정.
  - **ROE** = 순이익 / 자기자본 × 100
  - 일반적으로 10% 이상이면 우수한 것으로 평가돼.

---

3. 투자 리스크 평가 (안정성 분석)

- **현금흐름 분석 (Cash Flow Analysis)**: 기업의 실제 현금 창출 능력을 평가.
  - **영업활동 현금흐름**이 지속적으로 양수인지 확인.

▼ 영업 활동 현금 흐름

: 회사가 본업을 통해 벌어들인 실제 현금 흐름

$$\text{영업활동 현금흐름} = \text{순이익} + \text{비현금 비용(감가상각비)} - \text{운전자본의 변화}$$

- **순이익 (Net Income)**: 손익계산서에서 가져옴.
- **비현금 비용**: 감가상각비, 무형자산 상각비 등 (현금 지출이 없는 비용).

        - **운전자본의 변화**:
            - 운전자본 = 유동자산 - 유동부채
            - 증가하면 현금 유출(자산 증가), 감소하면 현금 유입(자산 감소).
    - **잉여현금흐름 (Free Cash Flow)** = 영업활동 현금흐름 - 자본지출(CAPEX)
    - 잉여현금흐름이 플러스면 긍정적.
- **부채 상환 능력**: 회사의 현금흐름으로 부채를 갚을 수 있는지 평가.
    - **부채비율**, **이자보상비율**, 그리고 **자기자본비율**을 함께 분석.
    - 자기 자본 비율 : 높을수록 안정적이며, 50% 이상이면 좋음.

$$\text{자기자본비율} = \frac{\text{자기자본}}{\text{총자산}} \times 100$$

▼ 분석 방법

- **부채비율이 높더라도 이자보상비율이 충분히 높으면 안전할 수 있음.**
- **자기자본비율이 높으면 부채비율이 높아도 위험성이 낮음.**
- 세 가지를 함께 분석하여 **안정성과 위험성을 동시에 고려해야 함**.

- **산업 및 시장 리스크**: 외부 요인도 고려해야 함. (정책 변화, 경쟁 상황, 기술 변화 등)

  : 웹스크래핑으로 산업 및 시장 리스트 뉴스들 가져오

---

4. 적정 주가 평가 (Valuation, 가치 평가)

- **PER (Price to Earnings Ratio, 주가수익비율)**:
    - **PER** = 주가 / 주당순이익(EPS)
    - 낮을수록 저평가된 것으로 볼 수 있지만, 성장주의 경우 높은 PER도 ㄱㅊ
- **PBR (Price to Book Ratio, 주가순자산비율)**:
    - **PBR** = 주가 / 주당순자산(BPS)
    - PBR ≤1 : 자산 가치보다 저평가된 것일 가능성이 있음.
- **EV/EBITDA (Enterprise Value to EBITDA Ratio) :**

- 기업 가치(EV)를 EBITDA로 나눈 값으로, 이 수치가 낮을수록 상대적으로 저평가 된 것일 수 있음.

▼ 계산 방법

EV (Enterprise Value, 기업 가치): 회사가 인수될 때 실제로 지불해야 하는 총 금액.

$$\text{EV} = \text{시가총액} + \text{총부채} - \text{현금 및 현금성 자산}$$

EBITDA : 기업의 본업에서 발생하는 수익을 측정하는 지표 (비용 구조를 비교할 때 유용).

$$\text{EBITDA} = \text{영업이익} + \text{감가상각비} + \text{무형자산상각비}$$

DCF (Discounted Cash Flow, 할인 현금흐름 모델):

- 미래의 현금흐름을 현재 가치로 환산하여 기업의 가치를 평가.
- 가장 정교하지만, 많은 가정이 필요함.

▼ DCF 계산 방법

- DCF는 **미래의 현금흐름을 현재 가치로 환산하여 기업의 가치를 평가하는 방법**

$$\text{DCF} = \sum_{t=1}^{n} \frac{FCF_t}{(1+r)^t} + \frac{TV}{(1+r)^n}$$

- FCFt : 특정 연도의 **잉여현금흐름 (Free Cash Flow)**.
- r : 할인율 (일반적으로 WACC, 가중평균자본비용).
- n : 평가 기간 (예: 5년 또는 10년).
- TV : **Terminal Value**, 평가 기간 이후의 가치를 추정.

- **DCF 분석에서 중요한 것**:
    - 정확한 **FCF 예측**과 적절한 **할인율 설정**이 필요.
    - 터미널 밸류 계산 방식도 여러 가지가 있음 (Gordon Growth Model 등).

5. 의사결정의 근거 제공 (종합 분석)

모든 지표를 종합하여 투자 결정을 내릴 때 고려할 점:

- **종합 점수화 또는 랭킹화**: **AHP, TOPSIS 같은 다기준 의사결정 기법을 사용해서 여러 지표를 통합할 수도 있음.**
- **업종 평균과의 비교**: 같은 산업 내 다른 기업들과의 비교가 중요함.
- **트렌드 분석**: 단일 시점의 데이터보다는 **여러 기간의 추세를 보는 것이 더 의미 있음.**
- **정성적 요소의 고려**: 브랜드 가치, 기술 혁신, 경영진의 능력 등도 분석에 포함할 필요가 있음.

▼ (서희정) 재무상태표, 손익 계산서, 현금 흐름표에서 우리가 추출할 항목들

1. 재무상태표 (Balance Sheet)

📄 추출할 항목들

- **유동자산 (Current Assets)**
    - 예: 현금 및 현금성 자산, 매출채권, 재고자산, 기타유동자산 (미수금, 선급비용, 유동금융자산)
- **유동부채 (Current Liabilities)**
    - 예: 매입채무, 단기차입금, 미지급비용, 선수금, 유동금융부채
- **총부채 (Total Liabilities)**
    - 예: 유동부채 + 비유동부채 (장기차입금, 사채, 미지급법인세, 확정급여부채)
- **자기자본 (Total Equity)**
    - 예: 자본금, 자본잉여금, 이익잉여금, 기타포괄손익누계액, 자사주
- **재고자산 (Inventory)**
    - 유동자산의 일부로, 당좌비율 계산에 필요
- **현금 및 현금성 자산 (Cash and Cash Equivalents)**

2. 손익계산서 (Income Statement)

📄 추출할 항목들

- **매출액 (Revenue, Sales)**
  - 예: 제품 판매, 서비스 제공, 로열티 수익, 투자수익 (지분법 이익, 배당금)
- **영업이익 (Operating Income)**
  - 매출액 - 영업비용 (매출원가, 판매비, 관리비 등)
- **순이익 (Net Income)**
  - 최종적인 이익으로, 모든 비용(영업비용, 이자비용, 세금 등)을 제외한 금액
- **이자비용 (Interest Expense)**
  - 금융 비용으로, 손익계산서 주석에 포함될 수 있음
- **감가상각비 (Depreciation)**
  - 비유동자산의 감가상각에 따른 비용
- **무형자산 상각비 (Amortization)**
  - 무형자산의 가치 감소에 따른 비용

3. 현금흐름표 (Cash Flow Statement)

📄 추출할 항목들

- **영업활동 현금흐름 (Operating Cash Flow)**
  - 본업을 통해 발생한 현금흐름 (예: 매출 현금 유입 - 영업비용 현금 유출)
- **잉여현금흐름 (Free Cash Flow, FCF)**
  - 영업활동 현금흐름 - 자본지출 (CAPEX)
- **자본지출 (CAPEX)**
  - 설비 투자, 건물 등 비유동자산의 취득 비용
- **재무활동 현금흐름 (Financing Cash Flow)**
  - 예: 배당금 지급, 차입금 변동, 주식 발행, 자사주 매입 및 처분
- **투자활동 현금흐름 (Investing Cash Flow)**
  - 예: 설비 투자, 자산 매각, 유가증권 매매, 비유동자산 취득

4. 외부 데이터

- **발행주식수 (Shares Outstanding)**
- **주가 (Stock Price)**
- **시가총액 (Market Capitalization)** = 주가 × 발행주식수
- **할인율 (Discount Rate, r)**

▼ (서희정) 선행기술 관련 논문

초등사회과 경제교육에서 금융이해력 향상을 위한 SEC 금융교육 프로그램 개발 및 적용

https://www.dbpia.co.kr/journal/detail?nodeId=T16854898

다문화가정 아동을 위한 온라인 금융교육프로그램 개발

https://www.dbpia.co.kr/journal/articleDetail?nodeId=NODE11633844

가치기반 아동금융교육 프로그램 개발 및 효과평가 : 개인재무관리 영역별 가치를 중심으로

https://www.dbpia.co.kr/journal/detail?nodeId=T15827669

검색

https://www.dbpia.co.kr/search/topSearch?searchOption=all&query=금융교육+프로그램+개발

노인소비자 대상 소비자금융 교육프로그램 개발

https://www.dbpia.co.kr/journal/detail?nodeId=T13991454

▼ (이설후) 챗봇구현

2. 구현 방법

2.1. 텍스트 추출 & 전처리

1. **드래그 & 드롭 혹은 텍스트 입력 이벤트**
    - 브라우저 상에서 사용자가 텍스트를 드래그한 뒤, 선택한 영역을 “드롭존” 혹은 “채팅 입력창” 등에 놓으면 해당 텍스트를 서버 또는 클라이언트 측 스크립트로 전달받는다.
    - 예: React에서 onDrop 이벤트, JavaScript FileReader 등을 활용 가능.
2. **전처리 및 필터링**

- 사용자가 드래그해 넣은 텍스트가 재무제표 정보인지, 회사 소개인지 등 간단한 분류를 할 수 있음.
- 텍스트 길이가 너무 길면 요약(summarization) 과정을 거쳐 “챗봇이 이해하기 적절한 길이”로 축소.
- 재무제표 숫자나 재무 지표, 전문 용어 등은 **토큰화** 또는 **키워드 추출** 기술로 걸러내 별도의 필드로 보관하면 이후 AI 설명 시 더욱 정확한 설명이 가능해짐.

2.2. AI 모델 연동

1. **사전 학습된 대규모 언어 모델(LLM) 활용**
    - 예: OpenAI의 GPT-3.5 / GPT-4, 혹은 Hugging Face Transformers 기반 모델 등.
    - 직접 모델을 호스팅할 수도 있지만, 초기에 빠른 프로토타입을 위해서는 API 기반 접근(예: OpenAI API)이 훨씬 편리함.
2. **프롬프트 엔지니어링(Prompt Engineering)**
    - 사용자 입력 텍스트와 함께, “이 텍스트를 초심자/청소년 대상 금융 교육용으로 쉽게 풀어서 설명해 달라”, “추가적으로 적절한 예시를 들어 달라” 등의 **지시(prompt)**를 함께 전달한다.
    - 지표 해석이라면, “이 텍스트 내에서 ROE, PER, PBR 등 용어가 등장하면 간단하고 쉬운 예시를 들어달라”와 같은 시스템 메시지를 미리 구성할 수 있음.
    - 꼬리질문을 유도하기 위해, 예: “궁금한 부분이 있으면 질문을 유도해 달라”는 식의 프롬프트를 추가한다.
3. **체크포인트 / 파인튜닝(Fine-tuning) / 임베딩(Embedding) & 벡터 DB**
    - **(선택)** 프로젝트에서 사용하는 용어 정의나 회사별 재무정보 등 **도메인 데이터**가 풍부하다면, 이를 **벡터 DB(예: Pinecone, Weaviate, Milvus 등)**에 임베딩 형태로 저장해 두고, 사용자가 드래그한 내용과 유사도가 높은 자료를 검색 후 모델에 전달(“Retrieval-Augmented Generation(RAG)” 방식).
    - 이렇게 하면 **도메인 특화된** 정확한 답변을 생성할 수 있음.
    - 또는 Hugging Face 모델을 **파인튜닝**하여 반복 질문에도 안정적으로 답변하도록 학습 가능(단, 데이터 준비와 리소스가 필요).

2.3. 대화 플로우 설계

1. **사용자 텍스트 입력 → AI 응답**
    - AI 응답을 받은 후, 클라이언트(웹/앱)에서 해당 응답을 표시.

- 이때 단순히 “답변”만 주는 것이 아니라 **‘계속 질문하기’** 혹은 **‘좀 더 쉽게 설명해 달라’** 같은 버튼을 제공해, 유저가 2차, 3차 꼬리질문을 던질 수 있도록 함.

2. **대화 컨텍스트 유지**
    - 꼬리질문에 대해 AI가 맥락을 유지하도록 하기 위해, 프롬프트에 “이전 대화 내용” 또는 “세션 히스토리”를 포함.
    - 대화형 API 사용 시, **대화 히스토리**를 일정 길이(토큰 제한)에 맞춰 관리해야 함(너무 길면 API 호출 비용 및 처리 속도 문제).
3. **사용자 난이도/레벨 조절**
    - 예: 초등학생, 중학생, 고등학생 레벨별로 다른 난이도의 단어를 사용하도록 모델에 지시 가능.
    - “설명을 쉽게 / 더 깊게 / 수학 공식까지 상세히” 등 선택지를 제공하면, 모델에 적절한 파라미터(temperature, max_tokens 등)나 프롬프트를 달리 줄 수도 있음.

---

3. 사용 기술 및 스택

3.1. 프론트엔드

- **React / Vue / Angular** 중 하나
- 드래그 & 드롭 기능 구현 시 React-Beautiful-DND 또는 HTML5 drag/drop API 사용 가능
- 채팅 인터페이스 구성 시 React-Chat-Widget, ChatGPT 스타일 UI 라이브러리 등 활용 가능

3.2. 백엔드

- **Node.js**(Express, NestJS 등) 또는 **Python**(FastAPI, Django) 등으로 API 서버 구성
- AI 모델 연동(OpenAI API 등) 기능 구현
- 사용자 인증, 권한 관리(만약 청소년 및 교사 계정이 있다면), 데이터베이스 연동

3.3. AI 모델 연동

1. **OpenAI API**
    - `text-davinci-003`, `gpt-3.5-turbo`, `gpt-4` 등의 엔진 사용

- API 키 관리, 프롬프트 엔지니어링, 토큰 관리(요금 계산)

2. **Hugging Face + Transformers**
   - 직접 모델 호스팅 시, GPU/메모리 자원 필요
   - Flask나 FastAPI로 Serving 구축 후 API로 제공 가능
3. **LangChain / Haystack 등**(선택)
   - Retrieval-Augmented Generation 파이프라인 편리하게 구성
   - 벡터 DB와 AI 모델의 질의응답 연결

3.4. 데이터베이스

- **SQL**: MySQL, PostgreSQL 등 -> 계정 관리, 히스토리 저장
- **NoSQL**: MongoDB 등 -> 대화 히스토리나 세션 데이터 기록
- **벡터 DB**(선택): Pinecone, Weaviate 등 -> 회사 재무제표 정보, 용어 사전 등 대규모 텍스트를 임베딩 형태로 저장 후 검색

---

4. 구현 난이도 & 고려사항

1. **난이도(중 ~ 상)**
   - 단순 OpenAI API 연동으로 “질문 → 답변” 정도의 MVP(최소 기능 제품)는 구현 난이도가 중간 정도.
   - 그러나 **정확도**(특히 금융, 재무 관련)는 도메인 지식과 정확한 데이터 업데이트가 필수.
   - “꼬리질문” 시 맥락 유지와 대화 히스토리 관리가 필요해 **프롬프트 엔지니어링**과 **대화 세션 설계**가 중요함.
2. **모델 선택**
   - SaaS(Software as a Service) 형태로 OpenAI를 쓰면 GPU 인프라 없이 손쉽게 시작 가능하지만, API 비용과 데이터 보안(개인정보, 민감 데이터)에 유의.
   - 사내에 GPU 인프라가 있고, 전문 인력이 있다면 Hugging Face 모델 호스팅 고려 가능(초기 세팅 난이도 높음).
3. **데이터 보강**
   - 재무제표, 투자 용어, 회사별 정보 등 국내 상황에 맞는 **한글 데이터**를 충분히 준비해야 함.

- AI가 틀린 정보를 “헛소리(Hallucination)”로 말할 수 있으므로, RAG(데이터 검색 → 답변 생성) 방식을 사용하면 정확도를 높일 수 있음.

4. **사용자 경험(UX)**
   - 청소년이 사용하는 만큼 **UI는 직관적으로**, **용어는 쉽게** 설명해야 함.
   - 특정 용어를 클릭하면 해당 용어의 정의로 이동할 수 있는 링크나 팝업 기능 등을 챗봇 대화창과 유기적으로 연결해주면 학습 효율이 올라감.
5. **추가 기능**
   - **토론 모드**: AI와 1:1로만 질문/답변이 아니라, 다른 학생/사용자도 함께 볼 수 있는 공개 토론 방식을 지원할 수도 있음(추가 난이도).
   - **학습 이력 저장**: 사용자가 질문한 내용, 봤던 답변들을 저장하고, 나중에 재접속 시 이어서 학습 가능하게 함.

---

5. 요약 및 제안 로드맵

1. **1차 프로토타입**
   - 프론트엔드(React 등)에서 텍스트 드래그 & 드롭 → 백엔드(Node/Python)에서 OpenAI API 호출 → 답변 표시
   - 세션/대화 컨텍스트는 간단히 구현
2. **2차 고도화**
   - 벡터 DB 사용해 도메인 지식(재무제표 용어집, 회사 데이터) 통합
   - 한글 투자 용어, 국내 재무제표 분석 특화
3. **3차 확장**
   - 꼬리질문, 난이도 조절, 사용자별 수준 맞춤형 답변
   - UI/UX 최적화(링크로 상세 설명 이동, 퀴즈 기능, 학습 수준 평가 등)

▼ # 시각화 자동화 코드

```
import requests
import pandas as pd
import matplotlib.pyplot as plt

def fetch_dart_data(api_key, corp_code, year, report_code="11011"):
    """ OpenDART API에서 데이터를 받아오는 함수 """
    url = "https://opendart.fss.or.kr/api/fnlttCmpnyIndx.json"
```

```
    params = {
        "crtfc_key": api_key,
        "corp_code": corp_code,
        "bsns_year": year,
        "reprt_code": report_code
    }

    response = requests.get(url, params=params)

    if response.status_code == 200:
        return response.json()
    else:
        print(f"Error {response.status_code}: {response.text}")
        return None

def json_to_dataframe(json_data):
    """ JSON 데이터를 Pandas DataFrame으로 변환하는 함수 """
    if json_data and "list" in json_data:
        df = pd.DataFrame(json_data["list"])
        return df
    else:
        print("No data available")
        return None

def plot_data(df, x_col, y_col):
    """ DataFrame 데이터를 바탕으로 그래프 생성하는 함수 """
    if df is not None and x_col in df.columns and y_col in df.columns:
        plt.figure(figsize=(10, 5))
        plt.plot(df[x_col], df[y_col], marker='o', linestyle='-', color='b',
label=y_col)
        plt.xlabel(x_col)
        plt.ylabel(y_col)
        plt.title(f"{y_col} Trend")
        plt.legend()
        plt.xticks(rotation=45)
        plt.grid()
        plt.show()
    else:
```

```
        print("Invalid columns for plotting")

# 🔷 실행 예제
API_KEY = "YOUR_API_KEY"
CORP_CODE = "YOUR_CORP_CODE"
YEAR = "2023"

# 1️⃣ API 데이터 가져오기
json_data = fetch_dart_data(API_KEY, CORP_CODE, YEAR)

# 2️⃣ DataFrame 변환
df = json_to_dataframe(json_data)

# 3️⃣ 그래프 생성 (예시: 'thstrm_amount'을 'sj_div' 기준으로)
if df is not None:
    plot_data(df, x_col="sj_div", y_col="thstrm_amount")
```

▼ # 브리핑 코드

```
import requests
import pandas as pd
import matplotlib.pyplot as plt
from transformers import pipeline

def fetch_dart_data(api_key, corp_code, year, report_code="11011"):
    """ OpenDART API에서 데이터를 받아오는 함수 """
    url = "https://opendart.fss.or.kr/api/fnlttCmpnyIndx.json"
    params = {
        "crtfc_key": api_key,
        "corp_code": corp_code,
        "bsns_year": year,
        "reprt_code": report_code
    }

    response = requests.get(url, params=params)

    if response.status_code == 200:
        return response.json()
    else:
        print(f"Error {response.status_code}: {response.text}")
```

```
        return None

def json_to_dataframe(json_data):
    """ JSON 데이터를 Pandas DataFrame으로 변환하는 함수 """
    if json_data and "list" in json_data:
        df = pd.DataFrame(json_data["list"])
        return df
    else:
        print("No data available")
        return None

def generate_summary(df):
    """ DataFrame 데이터를 기반으로 AI 브리핑 생성 """
    summarizer = pipeline("summarization")

    # 텍스트 요약할 재무 정보 정리
    financial_text = "\n".join(
        f"{row['sj_div']}: {row['thstrm_amount']}원" for _, row in df.iterrows()
    )

    summary = summarizer(financial_text, max_length=100, min_length=30,
do_sample=False)
    return summary[0]['summary_text']

def generate_report(api_key, corp_code, year):
    """ 전체 프로세스 실행 및 보고서 생성 """
    json_data = fetch_dart_data(api_key, corp_code, year)
    df = json_to_dataframe(json_data)

    if df is not None:
        summary = generate_summary(df)

        print("\n📌 **자동 브리핑 보고서** 📌")
        print(f"📅 사업 연도: {year}")
        print("📊 주요 재무 데이터:")
        print(df[['sj_div', 'thstrm_amount']].to_string(index=False))
        print("\n📝 AI 요약 브리핑:")
        print(summary)
```

```
    else:
        print("❌ 데이터 없음")

# 실행 예제
API_KEY = "YOUR_API_KEY"
CORP_CODE = "YOUR_CORP_CODE"
YEAR = "2023"

generate_report(API_KEY, CORP_CODE, YEAR)
```

https://www.classting.com/

▼ (강병진) 철학, 정당성, 차별점

📌 프로젝트 철학

> “청소년도 스스로 경제와 투자를 이해하고 판단할 수 있어야 한다.”
>
> 기존의 금융교육은 주입식이거나 흥미를 유도하지 못하는 방식이 대부분이다. 우리는 청소년이 단순히 정보를 외우는 것이 아니라, 실생활과 연결된 금융 개념을 스스로 묻고, 이해하고, 판단해보는 경험을 하도록 돕고자 한다. 이를 위해 복잡한 UI나 용어 대신, 자연어로 대화하는 챗봇 인터페이스를 통해 쉽고 친숙한 방식으로 경제 개념과 재무지표를 설명받고 체험할 수 있도록 구성했다.

✅ 정당성

> 청소년 대상 금융교육은 많지만, 대부분 일방향적이고 파편화되어 있어 실질적인 이해로 이어지지 않는다.
>
> 강의 영상, 퀴즈, PDF 자료 등은 정보는 많지만 **맥락이 없고, 개인의 수준이나 궁금증에 따라 설명이 달라지지 않는다.** 우리는 이를 보완하기 위해 **AI 챗봇 구조를 적용해, 자연어 질문에 맞춤형으로 응답하고 실시간으로 지표를 분석해주는 상호작용 방식**을 선택했다. 이

> 는 금융 개념을 스스로 탐색하고 연결하게 해주며, 사용자의 질문에 따라 유동적인 학습 흐름을 제공할 수 있다는 점에서 교육적 정당성이 충분하다.

🧠 차별점

> 단순한 챗봇, 단순한 금융교육 플랫폼이 아니다.
>
> 우리는 오픈소스 자연어 처리 모델(HuggingFace)을 활용해 사용자의 문장에서 기업명, 지표명, 질문 의도를 파악하고, 내부 재무제표 데이터베이스와 연결해 실제 데이터를 기반으로 **의미 있는 분석과 해석을 제공**한다. 예를 들어 "삼성전자 PER이 18이면 괜찮은 거야?"라는 질문에, 챗봇은 업종 평균과 비교한 결과를 설명하며 단순한 정의가 아닌 **판단 근거 중심의 응답**을 제공한다. 이 구조는 기존 금융교육의 한계를 극복하면서도, 기술적으로도 오픈소스를 적극 활용해 확장성과 실용성을 갖춘다.

✅ ChatGPT랑 다른 점 (왜 이걸 따로 만드는가?)

1. 🎯 정확한 지표 해석 → 실제 데이터 기반

- ChatGPT는 개념은 잘 설명하지만,

  "삼성전자 PER이 18이면 어떤 상태냐?" 같은 질문엔 **실제 지표를 조회해서 비교**하진 않음

- → 우리 챗봇은 **기업 데이터 + 업종 평균**을 바탕으로 "고평가/저평가" 같은 **판단**을 해줌

---

2. 🧠 눈높이 + 설명 방식 통제 가능

- ChatGPT는 말투나 설명 수준이 **고정되지 않음**

  (초등학생이 물어봐도 어려운 용어 쓸 수 있음)

- 우리는 말투, 설명 방식, 예시 등을 완전히 **통제된 UX로 설계 가능**

→ 금융에 대한 배경지식이 없는 사용자 기준으로 최적화 가능

---

3. 🧩 오픈소스 기반 → 직접 튜닝/커스터마이징 가능

- GPT는 BlackBox임. 내부 로직/데이터 수정 불가
- 우리는 HuggingFace 모델, Spring 백엔드, 데이터 전부 우리가 컨트롤

  → 사용자 피드백에 따라 모델 튜닝, 응답 방식 수정 등 **지속 개선 가능**

🧨 한 줄 요약

> ChatGPT는 "똑똑한 백과사전"이고,
>
> **우리는 "진짜 재무데이터를 보고, 쉽게 설명해주는 튜터"임.**

▼ (강병진) 기능

✅ "챗봇 + 재무제표 해석 시뮬레이션 (기초)"

> ❗ 단순 정보 전달 X →
>
> "지표값 + 기준 비교" 기반으로
>
> **"좋다/나쁘다" 간단하게 말해주는 챗봇**

🎯 예시 흐름

> 👤 사용자: “삼성전자의 PER이 18인데 이거 괜찮아?”
>
> 🤖 챗봇: “동일 업종 평균 PER은 10이야. 삼성전자는 평균보다 높아서 고평가일 가능성이 있어.”

🧩 기능 요약 (MVP 기준)

기능	설명
재무지표 DB	기업명, PER, PBR, ROE, 업종 평균 같은 정적 데이터

챗봇 UI	기본 입력창 + 응답창 (react-chat-ui 등)
사용자 질문 파싱	"PER", "ROE", "좋아?", "높아?" 등 키워드 추출
판단 로직	기준값과 비교해서 “좋다/나쁘다/보통”으로 응답 생성
챗봇 응답	“업종 평균보다 높음 → 고평가일 가능성 있음” 식 설명

💬 차별성 요약 (교수님 설득용)

- 단순 QnA 챗봇이 아니라 **지표 + 기준 비교**로 분석된 답변 제공
- **실제 데이터 기반의 맞춤형 피드백** 구조
- UI/UX는 오픈소스 활용 → 챗봇 구조 설계에 집중
- 백엔드는 간단한 분석 로직이지만 **정량 판단 알고리즘 존재**

▼ (강병진) 토스증권이랑은 뭐가 다른가?

✅ [차이점]

1. 사용자 주도 vs 시스템 주도

- **토스**: 지표를 보여주고, 시스템이 “이건 저평가예요”라고 자동 분석
- **너네**: 사용자가 “이게 괜찮은 거야?”, “왜 고평가야?”, “이럴 때 사도 돼?”라고 **직접 질문**

  → 거기에 대해 **자연어 해석 → 맞춤형 피드백 제공**

2. 자연어 기반 인터랙션

- 토스는 **정해진 틀의 정보 제공**
- 너네는 사용자가 아무 질문이나 던져도 **의도 파악 + 지표 추출 + 의미 있는 판단 근거 제시**
  - 예: “요즘 반도체 섹터 어때?” → 관련 업종 평균, 흐름, 대표 기업 PER 등 뽑아서 답변
  - 예: “LG에너지솔루션 비싸?” → 배터리 업종 비교, PER/PBR 해석, 성장성 코멘트 등

3. 교육적 목적 강조 (판단력 훈련)

- 토스는 분석 보여주고 끝
- 너네는 **왜 그런 판단이 나왔는지 해석**, 더 나아가서

- **“이걸 보면 뭐가 중요한 포인트인지”** 알려줌
- 투자 초보가 **스스로 투자 판단 능력을 키우는 구조**

4. 확장성 (사용자 맞춤형 시나리오 가능)

- 예:
  - “나는 PER보다 성장률이 더 중요해” → 너네는 그 성향까지 반영 가능
  - “이 회사 작년에 흑자전환했는데, 이런 경우 PER은 어떻게 해석해야 돼?” → 상황별 맥락 포함한 응답 가능

🔑 한줄 요약

> 토스는 정보를 보여주는 서비스,
>
> **너네는 정보를 “이해시키고 판단 도와주는” 서비스.**

▼ (강병진) 다트포인트와 차별점

- **대상 및 목적**
  - *dartpoint.ai*: 주로 재무제표와 공시 데이터를 기반으로 기업 분석 및 자동 보고서를 제공
  - *우리 프로젝트 (INVestigator)*: 단순 분석 보고서 제공을 넘어, 사용자가 직접 질문하고 해석하는 대화형 AI 멘토로서 **투자 판단력 훈련**에 초점
- **인터랙션 방식**
  - *dartpoint.ai*: 정형화된 데이터와 리포트 제공, 사용자가 원하는 정보 선택 중심
  - *우리 프로젝트*: 자연어로 질문을 하면, 기업명/지표 추출 후 **실제 데이터와 업종 비교**를 통한 맞춤형 해석 및 판단 근거 제공
- **교육 및 사용자 경험**
  - *dartpoint.ai*: 분석 정보를 제공하지만, 교육적/해설적 측면은 상대적으로 약함
  - *우리 프로젝트*: 반복 질의-응답을 통해 사용자가 스스로 **투자 판단 능력**을 기를 수 있도록 설계, **투자 학습자**의 성장 지원
- **기술 응용 및 확장성**
  - *dartpoint.ai*: 주로 DART 데이터 등 공시자료에 의존한 분석

- *우리 프로젝트*: HuggingFace 기반 자연어 처리와 RAG 구조를 활용해 다양한 질문 유형에 대응하며, **맞춤형 투자 멘토링** 확장이 용이

즉, dartpoint.ai가 **정적인 데이터 분석 서비스**라면, 우리 프로젝트는 **대화형 AI를 통한 실시간 투자 판단 학습**에 초점을 맞추어 차별화하고 있음.

▼ (강병진) 대화 예시

[예시 대화: 투자 판단력 훈련 챗봇 “INVestigator”]

사용자:

"삼성전자 PER이 18이면 괜찮은 건가요?"

챗봇 (INVestigator):

"현재 삼성전자의 PER은 18로 나타납니다. 같은 반도체 업종의 평균 PER은 약 15 정도인데, 이는 삼성전자가 업종 평균보다 다소 높은 평가를 받고 있음을 의미합니다.

- **해석1:** 삼성전자가 미래 성장성에 대한 기대를 반영해 프리미엄이 붙은 상태일 수 있습니다.
- **해석2:** 그러나 업종 내 경쟁 심화나 글로벌 시장 불확실성을 고려하면, 고평가로 볼 여지도 있습니다.

  따라서 단순히 '괜찮다'고 단정짓기보다는, 현재 시장 상황과 기업의 미래 전략을 함께 검토할 필요가 있습니다."

사용자:

"그러면 이걸 어떻게 투자 판단에 반영해야 할까요?"

챗봇 (INVestigator):

"먼저, PER이 업종 평균보다 높은 이유를 파악하는 것이 중요합니다. 예를 들어:

- **성장 기대:** 삼성전자의 경우, 미래 기술 투자나 글로벌 경쟁력 강화 계획이 반영되어 있을 수 있습니다.
- **리스크 요인:** 반면, 반도체 시장의 경기 변동성, 원자재 가격 상승 등의 리스크도 존재합니다.

  따라서, 투자 판단 시에는 PER 수치 외에도 기업의 재무 건전성, 성장 전략, 업계 트렌드 등을 종합적으로 고려해야 합니다.

  추가적으로, 과거 실적 및 미래 전망 자료를 비교 분석하면 보다 신중한 결정을 내릴 수 있습니다."

사용자:

"그럼 지금 상황에서는 삼성전자가 고평가된 건가요?"

챗봇 (INVestigator):

"수치상으로는 삼성전자가 업종 평균보다 높은 PER을 보이고 있어, 일반적인 기준에서는 고평가 신호로 해석될 수 있습니다. 하지만 고평가 여부는 단순 비교만으로 판단하기 어렵습니다.

- **긍정적 측면:** 미래 성장 가능성과 기술 리더십이 반영된 결과일 수 있음
- **부정적 측면:** 단기 시장 변동성과 외부 리스크가 반영된 결과일 수도 있음

  따라서, 추가적인 재무 지표(예: EPS, PBR 등)와 시장 동향 분석을 함께 진행한 후 최종 판단하는 것이 좋습니다."

이 예시 대화는 단순히 정적인 정보를 제공하는 것이 아니라,

사용자가 스스로 투자 판단의 근거를 이해하고, 추가 분석 포인트를 학습할 수 있도록 돕는 구조임을 보여준다.

즉, 우리 챗봇은 **단순 정보 제공이 아닌, 투자 판단력을 향상시키는 멘토 역할**을 한다는 점이 차별화 포인트야.

▼ vs ChatGPT

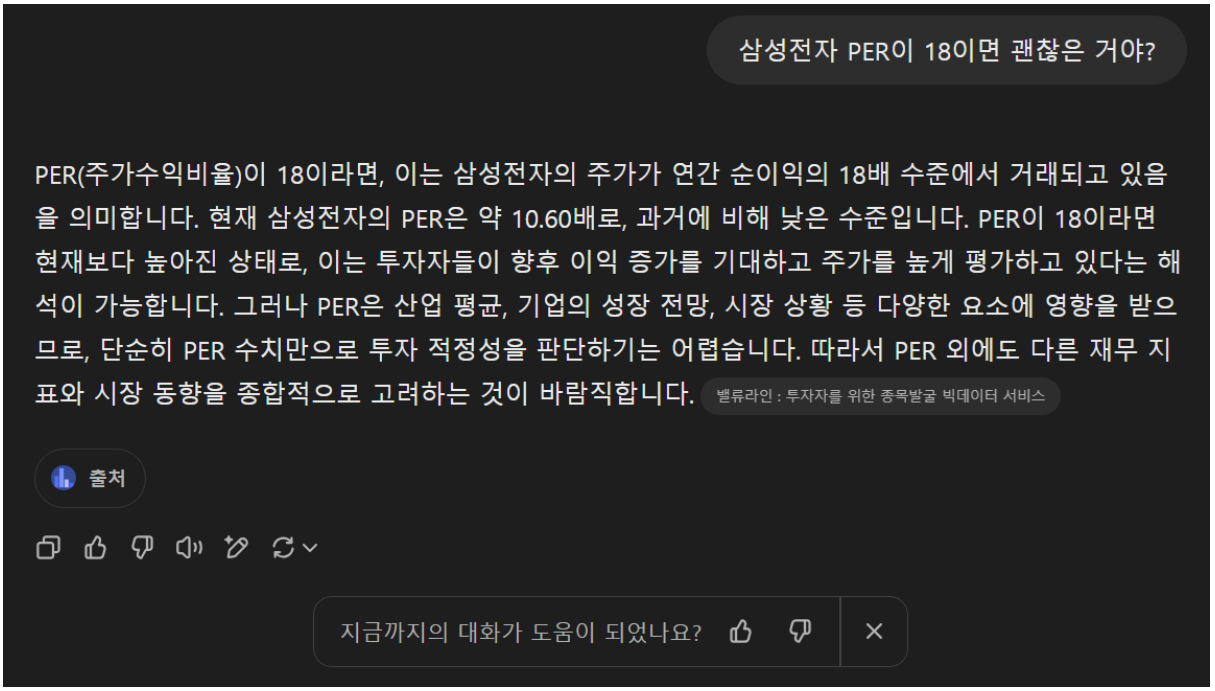
ChatGPT: 범용 대화형 AI로 투자 관련 질문에 대해 일반적인 정보와 정의 제공에 그침

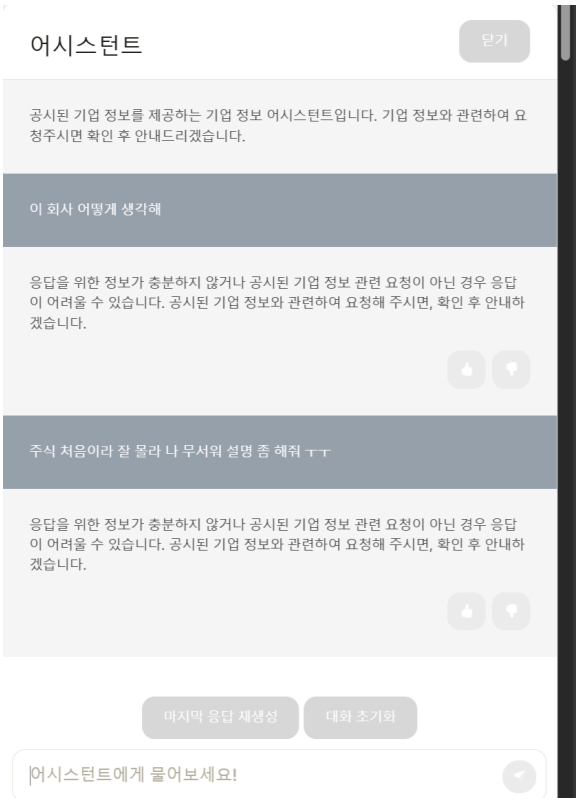
우리 프로젝트: "데이터 기반 해석 + 학습 피드백" 실시간 재무 데이터와 업종 비교를 기반으로, 투자 판단에 필요한 구체적 근거와 해석을 제공하며, 사용자가 스스로 판단력을 향상할 수 있도록 피드백을 제공함

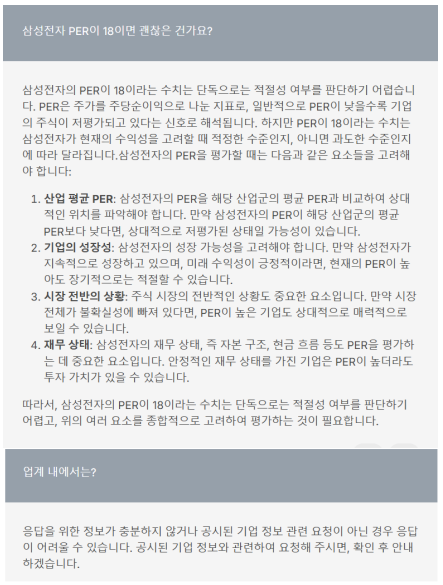
다트포인트: 정적인 공시 데이터와 재무제표 분석을 통해 자동 보고서를 생성하는 데 초점

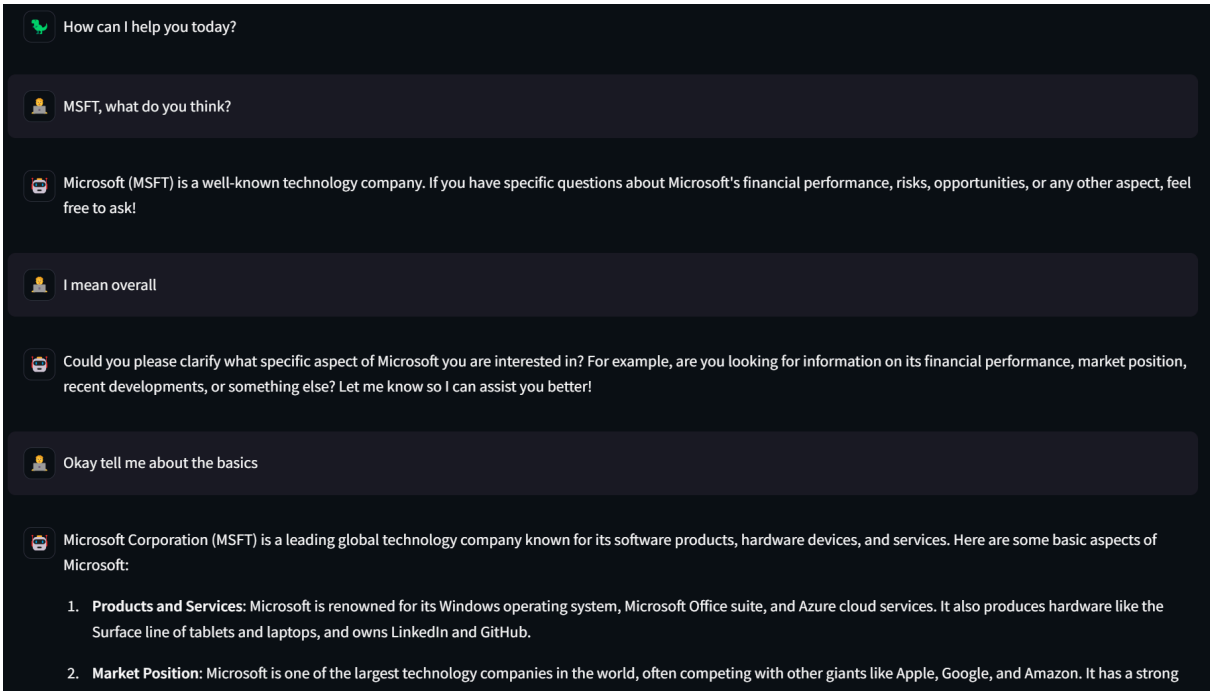
우리 프로젝트: 사용자가 자연어로 질문을 던지면, 해당 질문에 맞춰 내부 DB와 실시간 데이터를 연계해 맞춤형 분석과 업종 비교, 그리고 해석을 제공. 즉, 단순 정보 제공을 넘어 투자 판단력 향상을 위한 대화형 멘토 역할 수행

초보자가 가지는 어려움은 무엇인가? 자신에게 무엇이 부족한지, 무엇이 필요한지 잘 모른다는 점이다. 고수 투자자라면 어떤 정보를 어떤 소스에서 얻어야 할지, 어떤 관점을 택해야 할지 스스로 판단할 수 있다. 하지만 초보자는 정보의 홍수 속에서 막연하고 무섭다. INVestigator는 다양한 관점을 제공하며 투자자가 스스로 결정할 수 있는 판단의 근거를 제공하고 투자 감각을 길러주는 AI 멘토이다.


삼성전자 PER이 18이면 괜찮은 거야?
PER(주가수익비율)이 18이라면, 이는 삼성전자의 주가가 연간 순이익의 18배 수준에서 거래되고 있음을 의미합니다. 현재 삼성전자의 PER은 약 10.60배로, 과거에 비해 낮은 수준입니다. PER이 18이라면 현재보다 높아진 상태로, 이는 투자자들이 향후 이익 증가를 기대하고 주가를 높게 평가하고 있다는 해석이 가능합니다. 그러나 PER은 산업 평균, 기업의 성장 전망, 시장 상황 등 다양한 요소에 영향을 받으므로, 단순히 PER 수치만으로 투자 적정성을 판단하기는 어렵습니다. 따라서 PER 외에도 다른 재무 지표와 시장 동향을 종합적으로 고려하는 것이 바람직합니다.
밸류라인 : 투자자를 위한 종목발굴 빅데이터 서비스
출처
지금까지의 대화가 도움이 되었나요?


어시스턴트
닫기
공시된 기업 정보를 제공하는 기업 정보 어시스턴트입니다. 기업 정보와 관련하여 요청주시면 확인 후 안내드리겠습니다.
이 회사 어떻게 생각해
응답을 위한 정보가 충분하지 않거나 공시된 기업 정보 관련 요청이 아닌 경우 응답이 어려울 수 있습니다. 공시된 기업 정보와 관련하여 요청해 주시면, 확인 후 안내하겠습니다.
주식 처음이라 잘 몰라 나 무서워 설명 좀 해줘 ㅜㅜ
응답을 위한 정보가 충분하지 않거나 공시된 기업 정보 관련 요청이 아닌 경우 응답이 어려울 수 있습니다. 공시된 기업 정보와 관련하여 요청해 주시면, 확인 후 안내하겠습니다.
마지막 응답 재생성
대화 초기화
어시스턴트에게 물어보세요!


삼성전자 PER이 18이면 괜찮은 건가요?
삼성전자의 PER이 18이라는 수치는 단독으로는 적절성 여부를 판단하기 어렵습니다. PER은 주가를 주당순이익으로 나눈 지표로, 일반적으로 PER이 낮을수록 기업의 주식이 저평가되고 있다는 신호로 해석됩니다. 하지만 PER이 18이라는 수치는 삼성전자가 현재의 수익성을 고려할 때 적정한 수준인지, 아니면 과도한 수준인지에 따라 달라집니다.삼성전자의 PER을 평가할 때는 다음과 같은 요소들을 고려해야 합니다:
1. 산업 평균 PER: 삼성전자의 PER을 해당 산업군의 평균 PER과 비교하여 상대적인 위치를 파악해야 합니다. 만약 삼성전자의 PER이 해당 산업군의 평균 PER보다 낮다면, 상대적으로 저평가된 상태일 가능성이 있습니다.
2. 기업의 성장성: 삼성전자의 성장 가능성을 고려해야 합니다. 만약 삼성전자가 지속적으로 성장하고 있으며, 미래 수익성이 긍정적이라면, 현재의 PER이 높아도 장기적으로는 적절할 수 있습니다.
3. 시장 전반의 상황: 주식 시장의 전반적인 상황도 중요한 요소입니다. 만약 시장 전체가 불확실성에 빠져 있다면, PER이 높은 기업도 상대적으로 매력적으로 보일 수 있습니다.
4. 재무 상태: 삼성전자의 재무 상태, 즉 자본 구조, 현금 흐름 등도 PER을 평가하는 데 중요한 요소입니다. 안정적인 재무 상태를 가진 기업은 PER이 높더라도 투자 가치가 있을 수 있습니다.
따라서, 삼성전자의 PER이 18이라는 수치는 단독으로는 적절성 여부를 판단하기 어렵고, 위의 여러 요소를 종합적으로 고려하여 평가하는 것이 필요합니다.
업계 내에서는?
응답을 위한 정보가 충분하지 않거나 공시된 기업 정보 관련 요청이 아닌 경우 응답이 어려울 수 있습니다. 공시된 기업 정보와 관련하여 요청해 주시면, 확인 후 안내하겠습니다.


How can I help you today?
MSFT, what do you think?
Microsoft (MSFT) is a well-known technology company. If you have specific questions about Microsoft's financial performance, risks, opportunities, or any other aspect, feel free to ask!
I mean overall
Could you please clarify what specific aspect of Microsoft you are interested in? For example, are you looking for information on its financial performance, market position, recent developments, or something else? Let me know so I can assist you better!
Okay tell me about the basics
Microsoft Corporation (MSFT) is a leading global technology company known for its software products, hardware devices, and services. Here are some basic aspects of Microsoft:
1. Products and Services: Microsoft is renowned for its Windows operating system, Microsoft Office suite, and Azure cloud services. It also produces hardware like the Surface line of tablets and laptops, and owns LinkedIn and GitHub.
2. Market Position: Microsoft is one of the largest technology companies in the world, often competing with other giants like Apple, Google, and Amazon. It has a strong

수행 계획서 (4/2)

1. 프로젝트 수행팀 개요 (서)

- 수행 학기: 2025학년 1학기
- 프로젝트명: “INVESTigator – 성장하는 개인 투자자의 투자 감각을 키워주는 AI 멘토”
- 팀명: Spring

구분	성명	학번	소속학과	연계전공	이메일
팀장	서희정	2021110839	경제학과	데이터사이언스소프트웨어	pp123qq@dgu.ac.kr
팀원	강병진	2019111830	전자전기공학부	융합소프트웨어	2019111830@dongguk.edu
팀원	김시연	2020112503	산업시스템공학과	데이터사이언스소프트웨어	syeone01@dgu.ac.kr
팀원	이설후	2021113455	산업시스템공학과	데이터사이언스소프트웨어	2seolhoo@dgu.ac.kr

2. 프로젝트 수행계획

- 지도교수: SW교육원 이길섭 교수

2.1 프로젝트 개요 (이)

- 프로젝트 개요
  - 투자 초보자가 재무제표와 투자 지표를 쉽게 이해할 수 있도록 지원하는 AI 기반 챗봇을 개발한다. 어려운 투자 용어나 개념을 명확하게 설명하고 이해도를 높이는 것이 목표이다.
- 주제 선정 배경
  - 최근 투자 관심 증가로 초보 투자자들이 급증했으나, 복잡한 용어와 개념으로 인해 어려움을 겪고 있다. 이를 해결하고 누구나 쉽게 금융 개념과 재무 지식을 이해할 수 있도록 AI 기반의 개인 맞춤형 서비스를 제공하여 사회 전반의 금융 이해력을 높이고자 한다.
- 필요성 및 해결 방향
  - 투자 초보자들이 겪는 대표적인 문제점으로는 재무제표 해석, 투자 지표의 의미 파악 및 실제 활용이 어렵다는 점이다. 이러한 문제를 해결하기 위한 방안은 다음과 같다.
  - AI 자연어 처리 기술을 활용하여 사용자의 질문에 맞춤형 답변 제공
  - 기업의 재무보고서 및 기본 투자지표등 이해를 돕기 위한 쉬운 설명 및 다양한 실사례 기반 예시 제공
- 프로젝트 목적
  - 초보 투자자의 금융 지식을 향상시켜 올바른 투자 결정을 돕고, 개인의 금융 리터러시 능력을 향상시킨다.
- 최종 결과물
  - 본 프로젝트의 결과물은 다음과 같은 기능을 갖춘 사용자 친화적인 AI기반 챗봇 이다.
  - 투자 용어와 개념을 쉽게 설명하는 기능
  - 기업 재무 보고서의 주요 항목을 누구나 이해하기 쉽게 해석하고 핵심 지표를 시각화하여 전달하는 기능

2.2 추진 배경(자료조사 및 요구분석)

(1) 개발 배경 및 필요성 (서)

많은 투자자들은 양봉, 음봉과 같은 주가 그래프 분석 등 기술적 분석에 집중하는 경향이 있으나, 기업의 재무 상태를 정확히 파악하는 것이야말로 투자 판단의 핵심 요소라 할 수 있다. 최근 홈플러스의 사례는 이를 잘 보여준다.

홈플러스는 대형 세일 행사 '홈플런'과 모델 김수현을 내세운 마케팅 활동을 통해 긍정적인 반응을 얻었음에도 불구하고, 2025년 3월 기업회생 절차를 신청하였다. 이는 2018년 이후 지속된 매출 감소와 영업 적자, 그리고 높은 고정비 부담으로 인한 수익성 악화가 주요 원인이었다. 특히, 2024년 11월 말 기준 순차입금은 약 6.4조 원으로 현금 창출력 대비 과중한 재무 부담이 존재하였다.

이러한 재무적 어려움은 최근 발생한 문제가 아니며, 과거 재무제표 분석을 통해 그 위험 신호를 사전에 파악할 수 있었다. 그러나 많은 투자자들이 기술적 분석에 치중하는 경향으로 인해 이러한 위험 요소를 놓치는 경우가 빈번하다.

따라서, 투자자들이 기업의 재무 상태를 보다 쉽게 이해하고 분석할 수 있도록 지원하는 시스템의 필요성이 대두된다. 본 프로젝트는 투자 입문자들이 재무제표 분석을 효율적으로 학습하고 활용할 수 있도록 돕기 위해 **"투자 입문자 맞춤형 금융 학습 챗봇 시스템"** 을 개발하고자 한다. 이를 통해 투자자들이 보다 객관적이고 신뢰할 수 있는 투자 결정을 내릴 수 있도록 지원하는 것을 목표로 한다.

재무제표 분석의 중요성을 보여주는 상장 기업 사례들

1. **제너럴 모터스(General Motors, 2009년 파산 보호 신청)**

- 문제: 높은 부채 비율과 지속적인 적자 운영으로 인한 재정 악화.
- 재무제표 분석의 신호: 부채 비율 상승, 현금 흐름의 악화, 지속적인 영업 적자 등의 위험 신호가 재무제표에서 드러났음.
- 결과: 파산 보호 신청(챕터 11), 이후 구조조정을 통해 회생.

2. **한진해운(Hanjin Shipping, 2017년 파산)**

- 문제: 과도한 부채와 현금흐름 악화로 인해 경영 악화가 지속되었음.
- 재무제표 분석의 신호: 부채비율이 지속적으로 상승하고, 유동성 비율이 감소하는 등 재무 상태가 악화되었음.
- 결과: 파산 신청, 주요 노선의 운영 중단 및 자산 매각.

3. **도시바(Toshiba, 2015년 회계 부정 사건)**

- 문제: 도시바는 7년간 12억 달러 이상의 수익을 부풀린 회계 부정을 저질렀음.
- 재무제표 분석의 신호: 영업 이익률이 비정상적으로 높게 유지되는 등 불균형이 드러났음.
- 결과: 회계 부정이 밝혀진 후 주가 급락 및 기업 신뢰도 하락.

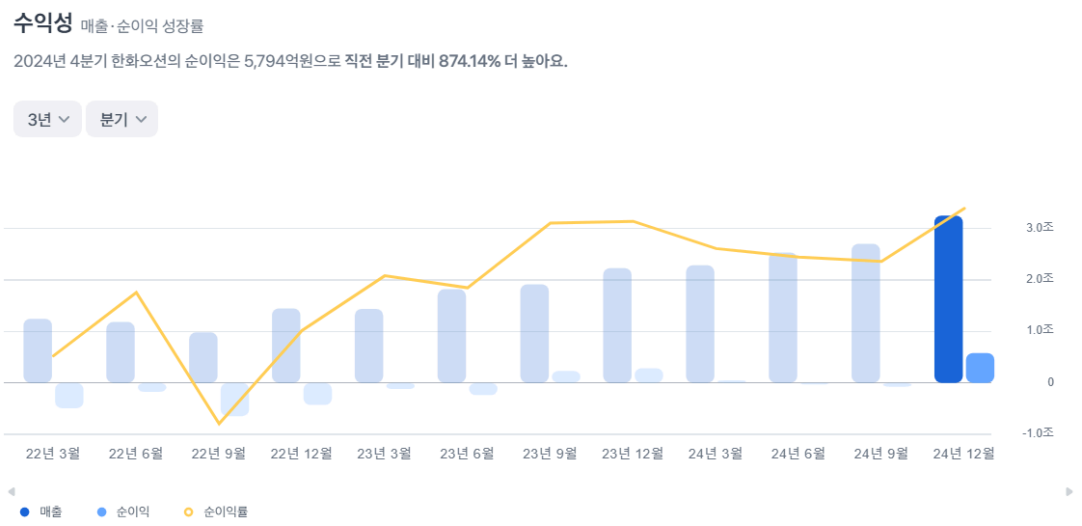
이와 같은 사례들은 재무제표 분석의 중요성을 잘 보여주며, 투자자들이 기업의 재무 상태를 면밀히 검토해야 함을 증명한다.

(2) 선행기술 및 사례 분석 (서)

- 토스증권

: 각 주식별로 재무 분석 그래프를 제공하여 투자자들이 기업의 재무 상태를 시각적으로 파악할 수 있도록 지원한다.

이러한 선행 사례는 투자자들에게 유용한 정보를 제공하지만, AI 챗봇과 같은 인터랙티브한 요소가 없어 사용자가 모르는 부분이 있을 경우 직접 추가적인 조사를 해야 하는 한계가 있다.



- DartPoint AI

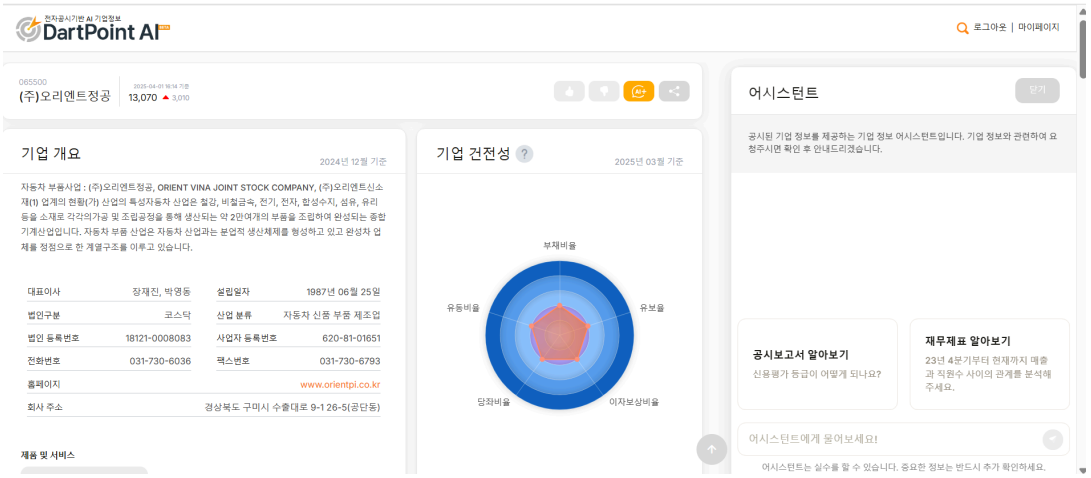
: DartOpen AI는 기업의 주요 지표, 재무 데이터, 공시 내 상세 정보를 손쉽게 확인할 수 있음.

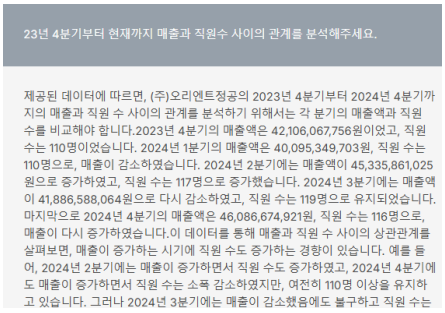
그러나 추천 질문(예: 특정 기업의 주요 지표 확인, 수익 구조 분석)에 대해서는 정확하게 응답할 수 있지만, 추천 질문 외의 사용자 질문에 대해 제대로 대응하지 못하는 경우가 많음.

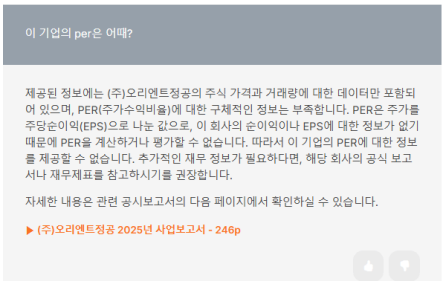
사용자가 궁금한 점을 자유롭게 질문하고 정확히 답변을 얻기 어려운 환경이 존재함.

특히, DartOpen AI는 **정확한 데이터베이스 연결 없이 공시 정보나 문서에서 정보를 추출하는 방식**을 주로 사용한다.

- PER 같은 지표는 실시간으로 변화하는 주가와 최근 실적을 반영해서 계산되기 때문에, 정확한 값을 제공하기 위해서는 최신 데이터베이스 접근이 필요하다.
- DartOpen AI는 과거 공시 자료만을 기반으로 정보를 제공해, 최신 PER 정보를 제공하지 못한다.







2.3 목표 및 내용

(1) 개발 목표 (강)

본 프로젝트의 목표는 단순한 정보 제공을 넘어, 청소년 등 초보 사용자가 실질적인 투자 판단력을 훈련할 수 있도록 돕는 AI 기반 주식 투자 멘토 시스템을 개발하는 것이다.
기존의 주식 관련 서비스들은 재무지표를 나열하거나, 종목 추천 정도의 정적인 기능에 그치며, 사용자의 투자 실력 향상에는 기여하지 못하는 한계가 있다. 우리는 AI 챗봇을 중심으로 한 UX 설계와 학습 피드백 시스템을 통해 사용자가 재무정보를 해석하고, 그에 따라 스스로 판단하도록 유도하는 새로운 형태의 투자 학습 경험을 제공한다.
이러한 투자 실력 훈련 중심의 차별화된 UX는 실제 시중 서비스에서 유사 사례를 찾기 어렵다는 점에서, 창의성과 실현 가능성을 동시에 갖춘 개발 목표이다.

(2) 개발 내용 (강)

■ 개발 범위 및 최종 결과물 구성
소프트웨어 형태의 웹 기반 프로토타입 개발

- Spring Boot 기반 백엔드 + React 프론트엔드 + AI API 연동 구조
- 주요 사용자 대상은 투자 입문 단계의 청소년(10대 후반 ~ 20대 초반)

■ 시스템 구성 및 기능
[최종 결과물: "AI 투자 멘토 웹 서비스"]

■ 핵심 기능
AI 챗봇 인터페이스　사용자가 재무지표(PER, PBR 등)나 종목에 대해 질문하면 단순 수치가 아닌 비교, 해석, 리스크를 포함한 피드백 제공
기업 정보 대시보드　주요 재무지표, 산업 평균과 비교, 그래프 시각화 등 시각 중심의 정보 구성
UX 기반 학습 피드백

- "왜 이 판단이 합리적인가?" 또는 "이 선택의 리스크는?" 등, 판단을 유도하는 질문-응답 기반 학습 설계

■ 시스템 특징 및 성능 수준
GPT API 기반 자연어 처리 기능 탑재 → 자유 질의/응답 가능

- 사용자의 투자 판단 이력을 분석하여 맞춤형 학습 피드백 제공
- 기존 주식 정보 앱 대비, 능동적 학습 경험 강화

항목	기존 AI 매매 프로그램	기업 정보 제공 사이트 (토스증권 등)	INVESTigator
목적	수익 극대화 / 자동 매매	지표 정보 제공 중심	투자 판단 훈련 / 실전 학습
기능 중심	"언제 살까?" 예측	"이 기업은 PER이 얼마" 등 정보 표시	"왜 이 수치가 중요한가?" 판단 보조
질문 방식	불가능 / 메뉴 선택	고정된 텍스트 기반 화면 제공	자연어 질의 가능 (ex. "이 정도면 싼 거야?")

설명 방식	없음 (기계적 판단)	정적 정보 (고평가/저평가만 표시)	AI가 지표 해석 + 업종 비교 + 판단 이유 설명
교육 효과	없음	낮음	반복 질문-응답을 통한 판단력 향상
확장성	특정 알고리즘에 의존	UI 한정적	오픈소스 기반 + 도메인 지식 추가 용이

(3) 대안 도출 및 구현 계획 (문과적으로만) (김)

- 선택된 설계문제의 해결방안(솔루션)과 최종 설계 결과물에 맞는 구현과 관련된 구체적인 계획을 서술한다.
  - 주요 기능을 구현하기 위한 방법
  - 서비스를 기능 구현을 위한 데이터 정의 및 자료 구조 정의
  - 전체 알고리즘에 대한 플로 차트

(5) 개발 환경 (김)

- 최종 설계 결과물의 구현 수단
  - 개발환경: 하드웨어 장비
  - 개발환경: 소프트웨어 툴, 언어 등※ 충분한 분량으로 개발환경에 대하여 상세하게 작성한다.

########## 대안 도출 및 구현

주요 기능을 구현하기 위한 방법

- 자연어 이해 및 처리 (OpenAI의 GPT 계열 또는 Lalma2 같은 LLM 활용, Spacy 또는 NLTK를 이용한 금융 용어 사전 구축)
- 재무 데이터 분석 및 시각화 (OpenDART API를 활용하여 기업 재무제표 데이터 수집, Pandas와 Numpy를 활용한 데이터 가공 및 분석, Matplotlib, Seaborn을 이용한 재무 데이터 시각화 기능)
- 대화형 챗봇 인터페이스 (Streamlit 또는 Gradio를 활용한 웹 기반 챗봇 UI 개발)

기능 구현을 위한 데이터 정의 및 자료 구조 정의

- 기업 재무 데이터 (예: 손익계산서, 대차대조표, 현금흐름표)
- 금융 용어 사전 (예: ROE, ROA, 부채비율 등 주요 재무 용어)
- 사용자 입력 데이터 (예: 사용자 질문, 질의 의도, 대화 내역)

전체 알고리즘에 대한 플로차트

1. 사용자 입력 처리 (텍스트 입력 → NLP 전처리 → 의도(intent) 분류)
2. 의도별 처리 로직 (OpenDART API에서 데이터 가져오기 → Pandas로 가공 → 시각화)
3. 답변 생성 및 출력 (템플릿 기반 응답 생성 → UI로 출력)

개발 환경

소프트웨어 툴 - 백엔드: FastAPI, Flask / 프론트엔드: Streamlit, Gradio / 데이터 저장소: SQLite, PostgreSQL
##################

2.4 기대효과 (이)

- 투자 초보자 및 일반 대중이 투자 용어와 개념을 쉽게 이해할 수 있어 금융 접근성 향상
- 개인의 금융 리터러시 향상으로 보다 안정적이고 합리적인 투자 의사결정 촉진
- 올바른 투자 정보 접근을 통해 경제적 손실 방지 및 경제적 자립도 증대
- 장기적으로 건전한 투자 문화 확산 및 금융시장 신뢰 제고를 통한 사회적 금융 안전망 강화

- 금융 교육 접근성 증대로 사회 전체의 금융 역량과 경제적 안정성 제고

2.6 참고문헌 (각자)

1. 저자, 제목, 교과목, ㅇㅇ대학, 연월.
2. 저자, 제목, 출판사, 연월.
3. 저자, 제목, 저널명, 권, 호, 연월, 페이지.
4. 저자, 제목, 사이트/블로그명, 링크, 연월.

New Proj Ideas

▼ (강) **돈쭐: 사용자 참여형 착한 소비 인증 챌린지 플랫폼**

1. 🎯 주제

사용자 참여형 착한 소비 인증 챌린지 플랫폼

> 사용자들이 선한 영향력을 가진 가게를 직접 제보하고,
> **영수증 인증을 통해 착한 소비를 기록·분석·시각화**하는
> 데이터 기반의 **참여형 착한 소비 플랫폼**

---

2. 🧠 사용할 오픈소스 / 라이브러리

- **OCR**
  - `Tesseract OCR`: 오픈소스 OCR 엔진
  - 또는 `Kakao OCR API` (더 정확함, 쉽지만 API 호출 비용 있음)
- **자연어 처리**
  - `KoNLPy` (Python) → 품목 키워드 추출용
  - 간단한 품목 분류 룰/사전 기반 매칭 가능
- **시각화**
  - `Chart.js`, `Recharts` (React용)
  - 월별 소비 히트맵, 카테고리별 점수 분포 등

---

3. 🔄 전체 프로세스

1. **착한 가게 제보**
   - 사용자: 가게 정보 + 착한 사유 등록
   - 관리자 or 커뮤니티 투표로 승인/거절
2. **가게 등록 및 분류**
   - 카테고리: 소상공인 / 친환경 / 공익적 가치 등

- 등록된 가게는 지도/리스트에 노출

3. **영수증 업로드**
   - OCR 처리 → 상호명 / 품목 추출
   - 등록된 착한 가게와 매칭되면 인증 완료
4. **점수 계산 + 피드백 메시지 + 기부**
   - 소비 카테고리 기반 점수 계산 (환경/지역/공익 점수)
   - “오늘 환경점수 2점 + 지역점수 1점 획득” 메시지 제공
   - 챌린지 누적 데이터 기록, 기부(네이버 콩처럼)
5. **대시보드 / 마이페이지**
   - 개인 소비 이력 시각화
   - 누적 점수, 뱃지, 카테고리별 소비 트리 제공
   - 챌린지 달성 현황 표시

---

4. ⚙️ 기술적 구현

기능	기술 스택 / 구현 방식
회원 관리	Spring Boot + JWT 로그인
OCR 처리	Python 서버 + Tesseract, React에서 이미지 업로드
착한 가게 제보/등록	DB + 관리자 페이지 구성
소비 분류	사전 기반 키워드 매칭 or 품목 분류 룰
점수 계산	카테고리 기반 점수 테이블 적용
피드백 메시지	소비 결과에 따라 동적 메시지 반환
마이페이지 / 시각화	Chart.js or Recharts, 개인별 소비 히스토리 표시

---

5. 🕒 예상 소요 시간

총 작업 기간: **3~4주 (팀 기준)**

> 기본 기능 + 간단한 시각화 + 챌린지 구현까지

단계	기간	메모
기획/역할 분배	2~3일	기능 범위, 페이지 정의

DB/ERD 설계	1~2일	가게, 소비, 사용자, 점수 테이블
백엔드 API 구현	5~7일	인증, 업로드, 처리, 점수 계산
프론트 UI 구성	4~6일	제보/업로드/마이페이지
OCR 연동 + 테스트	3~5일	Python + API 연결 or Tesseract
시각화/피드백 로직	2~4일	점수표 기반 계산 로직
마무리 디버깅/통합	3~4일	발표용 정리 포함

6. 📌 기존 사례 및 차별점

기존 사례

서비스	설명	한계점
우리동네 돈쭐내기	착한 가게 지도 제공	참여 구조 없음, 영수증 인증 없음
네이버 영수증 리뷰	오프라인 리뷰 + 포인트	가게 제보/착한 소비 분류 없음
당근마켓 동네가게	지역 가게 노출	착한 소비 개념 없음, 인증 구조 없음

우리 서비스의 차별점 요약

항목	기존 서비스	우리 서비스	차별점
가게 등록	기관 or 고정 DB	**사용자 제보 기반 동적 생성**	커뮤니티 기반 확장
인증 방식	없음 or 리뷰 기반	**OCR 기반 영수증 인증**	실제 소비 확인
리워드 방식	포인트 or 없음	**점수 + 피드백 + 챌린지 시각화**	동기 부여, 지속 사용 유도
기술 요소	정적 지도, 리뷰	**OCR, 점수 계산, 시각화, 사용자 히스토리**	기술적 깊이 보완

▼ 서희정

▼ 강아지와 함께하는 일정

🌟 주제: 반려견과 함께하는 하루 일정 짜기 (LLM 모델 활용)

반려동물과의 외출을 즐겁게 만들기 위해, **사용자가 원하는 시간을 입력하면 AI 모델이 맞춤형 하루 일정을 생성**해주는 서비스야. 예를 들어, 반려견과 3시간 동안 즐겁게 놀 수 있는 일정을 LLM 모델을 활용해 자동으로 구성해주는 거야.

✅ 목표 기능

- **사용자가 입력한 조건**에 따라 **AI 모델이 일정 제안하기.**
- 3시간의 시간을 **다양한 활동으로 나눠서** 제안하기.
- 사용자 맞춤형 추천 (예: 반려견의 크기, 나이, 활동 수준 등 고려).

🛠️ 구현 기술 스택

- **Frontend**: React (TailwindCSS로 스타일링)
- **Backend**: Flask (Python) 또는 FastAPI
- **AI 모델 (LLM)**: OpenAI GPT-4 또는 Hugging Face의 `text-davinci-003`, `gpt-3.5-turbo`
- **Database**: MongoDB (NoSQL) 또는 PostgreSQL (SQL) - 사용자 정보와 반려동물 프로필 저장
- **Deployment**: Vercel (Frontend) + AWS, Render, or DigitalOcean (Backend)

📌 기능별 상세 구현 방법

🔍 1. 사용자 입력 받기 (Frontend)

사용자가 입력할 정보:

- ⏰ **총 시간 (예: 3시간)**
- 📅 **활동을 나눌 방식 (예: 30분 x 6, 1시간 x 3 등)**
- 🐕 **반려동물 정보 (크기, 나이, 활동 수준 등)**
- 🎯 **선호하는 활동 종류 (산책, 공원 놀이, 실내 활동 등)**

```
// React (Next.js 또는 Vite 기반)

import React, { useState } from 'react';

const ActivityPlanner = () => {
  const [totalTime, setTotalTime] = useState(3); // 기본값 3시간
  const [segments, setSegments] = useState(6);   // 기본값 30분 x 6
  const [dogInfo, setDogInfo] = useState('');
```

```
  const [preferences, setPreferences] = useState('');

  const handleSubmit = async (e) => {
    e.preventDefault();

    const response = await fetch('/api/plan', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ totalTime, segments, dogInfo, prefere
nces })
    });

    const data = await response.json();
    console.log(data.plan);  // 일정 결과 확인
  };

  return (
    <div className="p-6 max-w-md mx-auto bg-white rounded-x
l shadow-md space-y-4">
      <h1 className="text-xl font-bold">반려견과 함께하는 하루 일
정 짜기</h1>
      <form onSubmit={handleSubmit} className="space-y-4">
        <div>
          <label>총 시간 (시간 단위): </label>
          <input type="number" value={totalTime} onChange={e
=> setTotalTime(e.target.value)} className="border p-1"/>
        </div>
        <div>
          <label>활동 분할 개수: </label>
          <input type="number" value={segments} onChange={e
=> setSegments(e.target.value)} className="border p-1"/>
        </div>
        <div>
          <label>반려동물 정보: </label>
          <textarea value={dogInfo} onChange={e => setDogInfo
(e.target.value)} className="border p-1 w-full"/>
        </div>
        <div>
```

```
                <label>선호하는 활동 종류: </label>
                <textarea value={preferences} onChange={e => setPref
erences(e.target.value)} className="border p-1 w-full"/>
            </div>
            <button type="submit" className="bg-blue-500 text-wh
ite p-2 rounded">일정 짜기</button>
        </form>
    </div>
  );
};

export default ActivityPlanner;
```

🔍 2. 백엔드 설정 (Flask & OpenAI API 연결)

```
# app.py (Flask)
from flask import Flask, request, jsonify
import openai

app = Flask(__name__)

# OpenAI API Key 설정
openai.api_key = "YOUR_OPENAI_API_KEY"

@app.route('/api/plan', methods=['POST'])
def generate_plan():
    data = request.json
    total_time = data['totalTime']
    segments = data['segments']
    dog_info = data['dogInfo']
    preferences = data['preferences']

    prompt = f"""
    나는 반려견과 함께하는 하루 일정을 짜주려고 해.
    총 {total_time}시간 동안 {segments}개의 활동으로 나누어 주세요.
    반려견 정보: {dog_info}
    선호하는 활동: {preferences}
```

```
    각 활동마다 구체적인 제안을 해주세요.
    """

    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[{"role": "user", "content": prompt}],
        max_tokens=500
    )
    plan = response.choices[0].message['content']

    return jsonify({"plan": plan})

if __name__ == '__main__':
    app.run(debug=True)
```

🔍 3. 프론트엔드 & 백엔드 연결하기

Frontend의 `fetch` 요청이 Flask API로 보내지고, 일정 결과를 받아서 표시하는 방식으로 동작함.

📌 구현 결과 예시 (LLM 모델로 생성된 일정)

사용자 입력:

- 총 시간: **3시간**
- 활동 분할: **30분 x 6**
- 반려동물 정보: **중형견, 2살, 활발함**
- 선호 활동: **산책, 공놀이, 물놀이**

LLM 모델 출력 (예시):

```
1. 09:00 - 09:30: 공원 산책하기 (가벼운 산책으로 준비 운동)
2. 09:30 - 10:00: 공놀이 (공원 내 넓은 공간에서 공 던지기)
3. 10:00 - 10:30: 강아지 전용 수영장 가기
4. 10:30 - 11:00: 수영 후 건조 및 간식 시간
5. 11:00 - 11:30: 반려동물 카페 방문하여 휴식
6. 11:30 - 12:00: 짧은 산책 후 귀가
```

📌 기술적 실현 가능성 평가

- ✅ **LLM 모델**로 텍스트 기반 일정 추천은 충분히 가능.
- ✅ Flask와 React를 연동하여 UI/UX 구현도 가능.
- ✅ 전체 일정 생성은 LLM 모델에 의존하므로 API 연결 속도만 최적화하면 됨.

🎯 다음 단계

- UI 개선 (일정 출력 결과를 깔끔하게 보여주기)
- 일정 추천 결과를 저장할 데이터베이스 연동 (사용자별 맞춤 데이터 유지)
- 활동 추천의 다양성 향상을 위해 모델 프롬프트 최적화

구현을 직접 해볼 수 있게 다음 단계로 도와줄까? 😊

▼ 이미지 생성 ai를 결합한 꿈 일기

네가 말한 **'사용자 개인화에 집중한 서비스 (페르소나 고정형 AI 생성)'** 를 구현하려면, 다음과 같은 단계로 개발을 진행하면 돼.

✅ 1. 사용자 페르소나 설정 (고정 주인공 설정)

📌 목적: 사용자가 지정한 외모나 캐릭터 스타일을 기반으로 일관성 있는 이미지 생성하기.

📌 방법:

1. **사용자 입력 데이터 수집**
    - 사용자에게 자신의 페르소나를 설정할 수 있게 UI 제공.
    - 사용자가 다음 중 선택하도록 함:
        - 📷 **사진 업로드** (본인의 얼굴이나 캐릭터의 이미지)
        - ✍️ **텍스트 기반 설명** (예: "긴 머리의 소녀, 푸른 눈, 몽환적인 스타일")
        - 🎨 **특정 스타일 선택** (예: 픽사 스타일, 애니메이션 스타일, 웹툰 스타일 등)
2. **사용자 페르소나 데이터 저장**

- 사용자가 설정한 페르소나 정보를 데이터베이스에 저장 (DB Schema 예시)

```
CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    username VARCHAR(50),
    persona_description TEXT,
    persona_image BLOB,
    style_preference VARCHAR(50)
);
```

- 이미지 기반 정보는 **파일 저장 시스템** 또는 **클라우드 스토리지 (예: AWS S3, Firebase Storage)** 에 저장 가능.

✅ 2. 페르소나 고정형 이미지 생성 모델 구축

📌 목적: 사용자 고유의 페르소나를 기준으로 AI 이미지를 일관되게 생성하기.

📌 방법:

1. **모델 선택**
   - OpenAI의 DALL·E, Stable Diffusion 또는 **DreamBooth (특정 캐릭터 학습에 특화된 모델)** 사용.
   - 예를 들어, **Stable Diffusion + DreamBooth** 조합으로 사용자 지정 스타일을 고정 가능.
     - Hugging Face 모델 예시: `stable-diffusion-v1-5`
     - DreamBooth 구현 예시: `dreambooth-hf`
2. **사용자 스타일 학습**
   - 사용자가 업로드한 이미지를 모델에 학습시켜서 **개인화된 스타일로 변환 가능하게 만듦.**
   - 예시 코드 (Python, `diffusers` 라이브러리 사용)

```
from diffusers import StableDiffusionPipeline
import torch
```

```
model_id = "CompVis/stable-diffusion-v-1-5"
pipe = StableDiffusionPipeline.from_pretrained(model_id, torch_
dtype=torch.float16)
pipe.to("cuda")

prompt = "A beautiful dream world with a character resembling
the user, in a fantasy style"
image = pipe(prompt).images[0]

image.save("user_dream_image.png")
```

3. **페르소나 유지**
   - 사용자가 생성할 때마다 **기존 페르소나 정보**를 prompt에 포함하여 모델에 입력.
   - 예시 프롬프트:

     > "A dream scene featuring [User's Persona Description], in a [User's Style Preference] style."

✅ 3. 사용자 경험 (UI/UX) 설계

📌 목적: 사용자 친화적인 인터페이스로 서비스의 차별성을 극대화하기.

📌 방법:

1. **페르소나 설정 화면** (초기 설정 또는 변경 가능)
   - 이미지 업로드, 텍스트 설명 입력, 스타일 선택 기능 제공.
   - 설정 후 미리보기 기능으로 사용자에게 결과를 보여줌.
2. **페르소나 기반 이미지 생성 화면**
   - 프롬프트 입력과 함께 **페르소나 고정 옵션 활성화** 버튼 제공.
   - 사용자별로 이전의 결과물도 확인할 수 있게 **갤러리 형태로 제공**.

✅ 4. 데이터베이스 설계 및 유지

📌 목적: 사용자가 언제든지 동일한 페르소나로 꿈을 생성할 수 있도록 데이터 유지.

📌 방법:

- 사용자별로 **고정된 페르소나 데이터를 저장**하기 위해 데이터베이스 활용 (SQL or NoSQL).
- 예를 들어, MongoDB를 사용한다면:

```
{
  "user_id": "12345",
  "username": "dreamer1",
  "persona": {
    "description": "긴 머리의 소녀, 푸른 눈, 몽환적인 스타일",
    "style_preference": "Anime",
    "image_url": "https://your-storage/user12345/persona.jpg"
  }
}
```

✅ 5. 웹/어플리케이션으로 구현하기 (두 달 안에 가능)

📌 핵심 기술 스택

- **Backend**: Flask (Python), FastAPI, or Django (Python)
- **Frontend**: React, TailwindCSS
- **AI 모델**: Stable Diffusion, DreamBooth
- **데이터베이스**: MongoDB (NoSQL), PostgreSQL (SQL)
- **스토리지**: Firebase Storage, AWS S3

📌 프로젝트 일정 (예상)

주차	작업 내용
1주차	기본 UI/UX 설계 및 Flask/FastAPI 설정
2주차	사용자 데이터베이스 구축 및 페르소나 설정 기능 구현
3주차	AI 모델 학습 파이프라인 구축 및 연결
4주차	사용자 이미지 생성 기능 구현 및 테스트

5주차	사용자 갤러리 및 꿈 기록 기능 구현
6주차	디버깅, 최적화 및 배포 (웹 or 앱)
7주차	사용자 피드백 반영 및 기능 개선
8주차	최종 마무리 및 발표 준비

📌 추가 팁

- 만약 웹으로 구현한다면 **Flask + React 조합**으로 빠르게 구축 가능.
- 앱으로 만들고 싶다면 **Flutter** 또는 **React Native** 사용 추천.

구체적인 코드 구현 예시나 구조가 필요하면 알려줘! 😊

▼ 냉장고 사진으로 강아지 음식 만들어주는 ai 레시피 어플

💡 **아이디어: 냉장고 사진 찍으면 만들 수 있는 강아지 간식 & 식사 레시피 추천 앱**

🔥 아이디어 설명

1. **핵심 기능**
    - 사용자가 냉장고 사진을 찍어서 업로드하면, **이미지 인식 모델**이 재료를 파악하고,
    - *강아지의 상태(노견, 아픈 강아지, 활동량 많은 강아지 등)**에 맞는 간식이나 밥 레시피를 추천해줌.
    - 간단한 레시피, 건강에 좋은 재료 위주로 제공.
2. **특징**
    - 📷 **사진 인식 기반 추천** (냉장고 속 재료 분석)
    - 🐕 **강아지의 상태에 따른 맞춤 추천** (노견, 활동량 많은 강아지, 다이어트 중, 아픈 강아지 등)
    - 🍲 **간식과 밥을 구분하여 추천** (예: 아픈 강아지에게는 부드러운 식사 추천)
    - 💾 **사용자 기록 저장 및 분석** (사용자가 만든 레시피 기록 유지)

📌 구현 기술 스택

- **Frontend**: React Native (앱 개발), TailwindCSS (웹 버전 스타일링)

- **Backend**: FastAPI (Python)
- **AI 모델 (이미지 인식)**: YOLOv8, Google Vision API, OpenCV
- **LLM 모델 (텍스트 기반 추천)**: GPT-4, BERT 등
- **Database**: MongoDB (NoSQL) 또는 PostgreSQL (SQL)
- **Storage**: Firebase Storage or AWS S3 (사진 저장)
- **Deployment**: AWS EC2, Render, or Vercel

📌 구현 방법 (기능별로 정리)

🔍 1. 이미지 인식 기능 구현 (재료 인식)

사용자가 찍은 냉장고 사진에서 **재료를 인식**하는 단계.

📸 이미지 인식 모델 설정

- **YOLOv8 (Ultralytics)**: 객체 인식에 특화되어 있음.
- **Google Vision API**: 빠르고 정확하게 다양한 재료를 인식 가능.

예제 코드 (YOLOv8 사용)

```
from ultralytics import YOLO
from PIL import Image

# 모델 로드
model = YOLO("yolov8x.pt")  # YOLOv8 모델 (기존 훈련된 모델 사용)

# 이미지 불러오기
image = Image.open("fridge.jpg")

# 이미지 예측
results = model(image)

# 인식된 객체 출력
for result in results:
    print(result.names)
```

🔍 2. 사용자 정보 입력 및 설정

사용자가 자신의 강아지 정보를 입력하고 맞춤 추천을 받을 수 있게 설정.

사용자 정보 예시

- **반려견 나이**: (어린 강아지 / 노견)
- **활동 수준**: (활발 / 보통 / 적음)
- **건강 상태**: (다이어트 중 / 특정 질병 있음 / 일반)
- **선호하는 식사 종류**: (간식 / 밥 / 특별식)

데이터베이스 스키마 (MongoDB 예시)

```
{
   "user_id": "user123",
   "dog_info": {
      "name": "Bobby",
      "age": 7,
      "activity_level": "적음",
      "health_condition": "노견",
      "preferences": "부드러운 음식"
   }
}
```

🔍 3. 레시피 추천 기능 (LLM 모델 사용)

사용자가 찍은 사진과 강아지 정보를 바탕으로 LLM 모델이 레시피를 추천해 줌.

📌 모델 설정 (GPT-4 API 예시)

```
import openai

openai.api_key = "YOUR_API_KEY"

def get_recipe(ingredients, dog_info):
   prompt = f"""
   냉장고 속 재료: {', '.join(ingredients)}
   반려견 정보: {dog_info}
```

```
    위의 정보를 바탕으로 강아지를 위한 건강한 간식 레시피를 추천해줘.
    레시피는 간단하고 강아지에게 해로운 재료는 사용하지 않아야 해.
    """

    response = openai.ChatCompletion.create(
        model="gpt-4",
        messages=[{"role": "user", "content": prompt}],
        max_tokens=500
    )

    return response.choices[0].message['content']
```

🔍 4. 결과 출력 & 사용자 피드백 수집 (UI 구현)

사용자에게 레시피를 보여주고 저장하거나 공유할 수 있게 제공.

```
// React Native 예시 (TailwindCSS 사용)
import React, { useState } from 'react';
import { View, Text, TextInput, Button, Image } from 'react-native';

const RecipeApp = () => {
  const [recipe, setRecipe] = useState('');

  const handleGetRecipe = async () => {
    const response = await fetch('/api/get_recipe', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ ingredients: ['당근', '닭가슴살'], dogInf
o: '노견, 부드러운 음식 선호' })
    });
    const data = await response.json();
    setRecipe(data.recipe);
  };

  return (
    <View className="p-4">
      <Button title="레시피 가져오기" onPress={handleGetRecipe} /
>
```

```
        {recipe && <Text className="mt-4 text-black">{recipe}</T
ext>}
      </View>
    );
};

export default RecipeApp;
```

📅 개발 일정 (8주 프로젝트)

주차	작업 내용
1주차	이미지 인식 모델 학습 및 데이터셋 구축
2주차	Flask / FastAPI 서버 구축 및 API 연결
3주차	사용자 정보 입력 시스템 및 데이터베이스 구축
4주차	LLM 모델 연결 및 레시피 추천 기능 구현
5주차	사용자 UI/UX 디자인 및 피드백 시스템 추가
6주차	버그 수정 및 최적화
7주차	배포 및 사용자 테스트
8주차	최종 마무리 및 발표 준비

💡 차별화 포인트

- 📸 **사진 인식 기반 추천** (재료를 분석하여 자동 추천)
- 🐕 **개별 강아지 맞춤형 레시피 제공** (나이, 건강 상태, 선호도 반영)
- 📊 **사용자 기록 저장 및 분석** (사용자가 만든 레시피를 기록하여 피드백 제공)

다음 단계로 실제 코드를 작성하거나 앱으로 배포하는 방법까지 도와줄까? 😊

▼ 강아지 산책 틴더 어플

💡 **아이디어: GPS 기반 강아지 산책 평가 및 애견인 만남 앱**

🔥 아이디어 설명

1. **핵심 기능**
    - 📍 **GPS를 이용해 산책 경로 기록 및 평가하기.**

- 🦮 **강아지의 정보(크기, 활동 수준 등)에 따라 경로 평가 가능.**
- 🔔 **근처 애견인이 있으면 알림 기능으로 즉석 만남 기회 제공.**

2. **특징**
   - 경로 평가 기능을 활용해 **"대형견에게 적합한 산책로", "소형견에게 안전한 길"** 같은 정보를 제공.
   - 다른 애견인들과 **즉석으로 만남을 주선하여 커뮤니티 기능 제공.**
   - 사용자 기록을 데이터베이스로 분석해 **맞춤형 산책 추천 경로 제공.**

📌 구현 기술 스택

- **Frontend**: React Native (앱 개발), TailwindCSS (웹 버전 스타일링)
- **Backend**: FastAPI (Python)
- **GPS 트래킹**: React Native Geolocation API, Google Maps API
- **Database**: PostgreSQL or Firebase (사용자 데이터 및 경로 평가 저장)
- **실시간 알림 기능**: WebSockets (Socket.io), Firebase Messaging
- **AI 모델 (경로 추천 분석)**: GPT-4, Scikit-Learn (사용자 리뷰 분석)

📌 기능별 상세 구현 방법

🔍 1. GPS 기반 산책 경로 기록 (Frontend & Backend)

사용자가 앱을 켜고 GPS를 활성화하면 산책 경로를 기록함.

📲 React Native (Frontend) 예제

```
import React, { useState, useEffect } from 'react';
import { View, Text, Button, PermissionsAndroid } from 'react-nativ
e';
import Geolocation from 'react-native-geolocation-service';

const WalkTracker = () => {
  const [location, setLocation] = useState(null);
  const [route, setRoute] = useState([]);

  useEffect(() => {
```

```
    const requestLocationPermission = async () => {
      const granted = await PermissionsAndroid.request(
        PermissionsAndroid.PERMISSIONS.ACCESS_FINE_LOCATION,
      );
      if (granted === PermissionsAndroid.RESULTS.GRANTED) {
        Geolocation.watchPosition(
          position => {
            const { latitude, longitude } = position.coords;
            setLocation({ latitude, longitude });
            setRoute(prevRoute => [...prevRoute, { latitude, longitude }]);
          },
          error => console.log(error),
          { enableHighAccuracy: true, distanceFilter: 10 }
        );
      }
    };

    requestLocationPermission();
  }, []);

  return (
    <View>
      <Text>현재 위치: {location ? `${location.latitude}, ${location.long
itude}` : '위치 불러오는 중...'}</Text>
      <Button title="경로 저장" onPress={() => console.log(route)} />
    </View>
  );
};

export default WalkTracker;
```

🔍 2. 경로 평가 기능 (강아지 정보별로 평가 가능하게 설정)

- 사용자가 산책 후 경로에 대해 평가를 남길 수 있도록 설계.
- 평가 항목:
  - 🌳 경로의 안전성 (대형견/소형견 기준)
  - 🐾 편안함 (예: 포장이 잘 되어 있는지, 잔디가 많은지 등)

- 📏 거리 및 난이도 (짧음, 보통, 길음 / 평지, 오르막길 등)

🔍 3. 강아지 정보에 따른 맞춤 경로 추천 기능

- 사용자가 입력한 강아지 정보 (크기, 활동 수준, 건강 상태 등) 기반으로 경로를 추천.
- 사용자가 평가한 데이터 기반으로 추천 알고리즘 학습 가능 (Scikit-Learn, GPT-4 등).

📌 예제 데이터베이스 스키마 (PostgreSQL)

```
CREATE TABLE Users (
    user_id SERIAL PRIMARY KEY,
    username VARCHAR(50),
    dog_size VARCHAR(10),  -- 소형, 중형, 대형
    activity_level VARCHAR(20),  -- 활발함, 보통, 적음
    health_condition VARCHAR(50)
);

CREATE TABLE Routes (
    route_id SERIAL PRIMARY KEY,
    user_id INT REFERENCES Users(user_id),
    route_data JSONB,
    evaluation TEXT,
    recommended_for VARCHAR(50)
);
```

🔍 4. 근처 애견인 알림 기능 (실시간 만남 알림)

- 사용자가 산책 중일 때 **WebSockets (Socket.io)** 나 **Firebase Messaging** 으로 주변 사용자와 연결.
- 근처 반경 (예: 100m 이내)에 동일 앱을 사용하는 사람이 있다면 진동 및 알림 제공.

📌 예제 코드 (Socket.io)

```
from fastapi import FastAPI, WebSocket
from typing import List
```

```
app = FastAPI()
connected_users: List[WebSocket] = []

@app.websocket("/ws")
async def websocket_endpoint(websocket: WebSocket):
    await websocket.accept()
    connected_users.append(websocket)

    try:
        while True:
            data = await websocket.receive_text()
            for user in connected_users:
                await user.send_text(f"근처 사용자 발견: {data}")
    except:
        connected_users.remove(websocket)
```

🔍 5. 사용자 피드백 저장 및 분석

- 사용자가 평가를 남기면 이를 데이터베이스에 저장하고, 이후 경로 추천 시 활용.
- 리뷰 분석을 위해 간단한 NLP 모델 (Scikit-Learn) 활용 가능.

📅 개발 일정 (8주 프로젝트)

주차	작업 내용
1주차	GPS 트래킹 기능 구축, 사용자의 경로 기록 기능 구현
2주차	강아지 정보 입력 시스템 및 데이터베이스 구축
3주차	경로 평가 기능 및 사용자 피드백 시스템 구축
4주차	근처 사용자 알림 기능 구축 (Socket.io)
5주차	사용자 맞춤 경로 추천 알고리즘 개발
6주차	UI/UX 디자인 및 최적화
7주차	디버깅 및 배포 (앱스토어 or 웹 배포)
8주차	사용자 피드백 반영 및 발표 준비

💡 차별화 포인트

- 📍 **GPS 기반 경로 평가** (특정 강아지 정보별로 평가)
- 🔔 **실시간 애견인 만남 알림 기능** (커뮤니티 활성화)
- 🐾 **사용자 맞춤형 경로 추천 시스템** (리뷰 기반 학습 모델)

이걸 실제로 구현하려면 백엔드와 프론트엔드를 어떻게 연결하는지, 그리고 사용자 인터페이스를 어떻게 설계할지 알려줄까? 😊

▼ 이설후

▼ 여행기록

프로젝트 이름: **Travelog.AI – 너만의 감성 여행 플래너 & 기록 도우미**

🧭 프로젝트 개요

Travelog.AI는 사용자의 취향과 예산을 기반으로 여행 루트를 자동으로 생성해주고,
여행 후에는 사진과 간단한 기록만으로 감성적인 여행 회고록을 만들어주는 올인원 AI 여행 도우미입니다.

여행의 시작부터 끝까지 – **계획, 실행, 기록** – 모든 순간을 AI가 함께하며,
사용자는 단순히 여행을 '가는 것'이 아닌 '기억되는 경험'으로 남길 수 있습니다.

✨ 핵심 기능

구분	기능	설명
여행 전	여행 성향 분석	GPT가 사용자 취향(자연, 맛집, 휴식 등)을 파악하여 스타일 분류
여행 전	일정 & 루트 추천	GPT가 2박 3일 등 일정 구성 + 동선 고려하여 장소 제안
여행 전	지도 시각화	추천 경로를 Folium 기반 지도에 표시
여행 전	**예산 기반 루트 생성**	입력된 예산을 환율로 변환해 숙소/식사/교통/체험 항목별 소비 제안
여행 후	사진 업로드	여행 사진 여러 장 업로드 가능
여행 후	한 줄 기록 입력	각 사진별 한 줄 설명 (또는 자동 캡션 지원)
여행 후	감성 회고록 자동 생성	GPT가 스토리, 일기, 영상 자막 등 감성적인 문장 생성

기타	저장/공유 기능	회고록을 PDF로 저장하거나 소셜 공유 가능

🔁 전체 사용자 흐름 (User Flow)

1. 여행 전 단계
    - "나는 바다랑 맛집 좋아해!" + 예산 입력
    - AI가 성향 분석 + 추천 루트 제시 + 예산 계획까지 자동 구성
    - 지도와 표로 확인 → 확정 일정표 저장
2. 여행 후 단계
    - 사진 업로드 + 한 줄 메모 입력
    - AI가 감성적 문장으로 회고록 자동 생성
    - PDF 파일로 저장하거나 공유 가능

💡 예산 기반 루트 생성 – 세부 예시

> 입력: "2박 3일 일본 여행, 예산 700,000원"
>
> 💱 실시간 환율: 1엔 = 9.1원 → 총 ¥76,923
>
> ✈️ 여행 예산 분배:
>
> - 숙박: ¥30,000 (비즈니스 호텔)
> - 식사: ¥15,000 (평균 ¥1,500/식)
> - 교통: ¥5,000 (패스 포함)
> - 입장/체험: ¥10,000
> - 여유/쇼핑: ¥16,923

→ 그 예산 안에서 GPT가 최적의 루트와 활동 구성까지 제안

🛠️ 기술 스택

분야	기술

언어	Python 3.10+
프론트	Streamlit (MVP), 또는 React 확장 가능
백엔드	FastAPI (선택)
AI 엔진	OpenAI GPT API + LangChain
이미지 처리	Pillow, OpenCV
지도 시각화	OpenStreetMap + Folium
환율 API	exchangerate.host API (무료)
데이터 저장	SQLite / Firebase
회고록 출력	FPDF / ReportLab (PDF 저장용)

🎯 프로젝트의 차별점

기존 여행 서비스	Travelog.AI
단순 관광지 추천	사용자의 취향과 예산을 함께 고려한 일정 자동 생성
기록은 수동 작성	사진 + 메모만으로 AI가 감성 회고록 자동 생성
환율 반영 안 됨	실시간 환율로 예산 계획까지 구성
기록 & 계획 분리됨	여행 전후를 하나로 연결하는 완성형 경험 제공

👤 타겟 사용자

- 여행을 처음 가보는 대학생, 사회초년생
- 감성적인 여행 기록을 남기고 싶은 SNS 유저
- 귀찮은 걸 싫어하는 사람 (일정 & 기록 자동화)

🌱 향후 확장 아이디어

- 항공권 실시간 최저가 연동 (크롤링 or API)
- GPT 프롬프트 자동 최적화 (여행 스타일별 템플릿)
- 동영상 생성 기능 (사진 + 회고록 → 영상 자동화)
- 사용자간 일정 공유 기능 / SNS 연동

📝 요약

> Travelog.AI는 여행의 시작과 끝을 연결하는 감성+실용 통합 플랫폼으로,
>
> 사용자의 성향과 예산에 맞는 맞춤형 루트 생성과
>
> 여행 후 감성적인 회고록 자동화를 통해
>
> '여행을 완성된 추억'으로 바꿔주는 AI 서비스입니다.

▼ 강아지 산책일기

프로젝트 이름: **강아지가 바라본 세상**

반려견 시점 자동 그림 스토리북 생성기

🐾 프로젝트 개요

"강아지가 바라본 세상"은 사용자가 산책 중 찍은 강아지 사진을 업로드하면,
AI가 이를 시간 순서로 정리하고, 마치 강아지가 하루를 이야기하듯 따뜻한 그림일기를 자동으로 생성해주는 감성 중심의 웹 서비스입니다.

사진에 담긴 순간들을 AI가 해석해 동화처럼 풀어내며, 반려견과의 추억을 특별한 콘텐츠로 남길 수 있게 돕습니다.

✨ 핵심 기능

기능	설명
사진 업로드	사용자가 산책 중 촬영한 사진을 업로드
시간 순 정렬	EXIF 정보를 활용해 사진 순서 정리
키워드 추출	각 사진에 대한 간단한 설명 또는 자동 캡션 생성
스토리 생성	GPT가 강아지 시점에서 동화 형식 이야기 작성
일기 구성 출력	사진 + 설명 + AI가 쓴 이야기 통합 뷰 제공
다운로드 기능	그림일기 PDF로 저장 또는 공유 기능 제공

🧠 개발 방법 요약

1. **사진 업로드 + 메타데이터 추출**
    - 사용자가 여러 장의 사진 업로드 → EXIF 시간 정보 추출

2. **텍스트 정보 생성 (선택)**
    - 이미지 캡션 수동 입력 또는 CLIP 기반 추출
3. **GPT 프롬프트 구성**
    - 키워드 또는 캡션 기반으로 이야기 생성용 입력 구성
4. **GPT로 스토리 생성**
    - LangChain 활용해 이야기 구성 흐름 자동화
5. **스토리북 형식으로 렌더링**
    - 사진 + 이야기 + 날짜를 Streamlit 등으로 UI 구현
6. **PDF or 공유 기능 구현**
    - FPDF/ReportLab 사용해 파일 생성 가능

📊 전체 프로세스 요약

1. 사용자 사진 업로드
2. EXIF 정보 기반 시간순 정렬
3. 각 사진에 대해 수동 or 자동 캡션 생성
4. 캡션 기반 GPT 프롬프트 구성
5. GPT로 동화 형식 이야기 생성
6. 이야기와 사진을 결합하여 그림일기 형식 출력
7. 사용자가 PDF로 저장하거나 공유

⏱️ 예상 개발 소요 시간 (MVP 기준)

단계	기간	설명
요구사항 정리 & 기획	1일	기능 정리 및 역할 분배
기본 UI 구현	1~2일	Streamlit 기반 업로드/출력 화면 구성
EXIF 정렬 & 캡션 입력 처리	1일	이미지 정렬 및 키워드 수집 처리
GPT 연동 및 이야기 생성	1일	LangChain 구성 및 스토리 출력 확인

일기 렌더링 & 저장 기능	1~2일	PDF 출력 및 공유 기능 포함
테스트 및 시연 정리	1일	샘플 데이터로 결과물 준비
총계	**5~7일**	3~4인 팀 기준, MVP 수준 개발

🔍 현재 상용화된 유사 서비스와의 차이점

서비스명	기능 요약	차별점
Lollipop AI Diary	아기 영상 기반 AI 일기 자동 생성	강아지 전용 + 동화 스타일 없음
Childbook.ai	이야기 → 삽화 생성 기반 AI 동화 제작	텍스트 중심, 일기형 UX 아님
Recraft (AI Illustration)	사용자가 텍스트 입력 → 삽화 생성	사용자 프롬프트 기반, 반려동물 중심 아님

결론:

- 본 프로젝트는 **"사진 기반 + 반려동물 시점 + 동화 스타일 스토리 생성"**이라는 측면에서 **새롭고 감성적인 경험을 제공**하며,
- **일상 기록 + AI 창작 + 정서적 만족**을 동시에 충족하는 독창적 서비스입니다.

🧰 사용 기술 스택

분야	기술
언어	Python 3.10 이상
프론트엔드	Streamlit or React (MVP는 Streamlit 추천)
백엔드/API	FastAPI (선택)
AI 텍스트 생성	OpenAI GPT API + LangChain
이미지 처리	Pillow, OpenCV, EXIF
저장	Firebase / SQLite (간단한 저장용)
PDF 생성	FPDF, ReportLab (선택)

👥 타겟 사용자

- 반려견과의 추억을 특별하게 기록하고 싶은 보호자
- 반려동물 일상을 감성적으로 표현하고 싶은 일반 사용자

- 감성 기반 SNS 콘텐츠를 만들고 싶은 MZ세대

🌱 향후 확장 아이디어

- Stable Diffusion을 통한 동화 스타일 이미지 자동 생성
- 강아지 이름/성격에 따라 문체 변화 적용
- TTS 연동: 그림일기를 음성으로 읽어주는 기능
- 사용자별 "한 달 그림책 자동 완성 기능"

🎯 요약

> 강아지의 시선으로 바라본 세상을 이야기로 풀어주는,
>
> 나만을 위한 감성 그림일기 서비스
>
> https://blog.naver.com/ckdgus5290/222200219365

✅ AI 기술을 따뜻하게, 그리고 재미있게 활용하는 진짜 실용+감성 프로젝트!

▼ 김시연

- 이미지 생성 모델을 이용한 서비스
  이상형에 부합하는 연예인 찾기
  본인만의 부합하는 이상형을 묘사하고, 이상형에 맞는 그림을 이미지화하고,
  그 이미지에 맞는 연예인을 찾아주는 서비스?

4/7 회의록

태그	팀회의
날짜	@2025년 4월 7일

▼ (강) 백엔드

1.백엔드 개발 담당
<할 일:>
• 기본 API 설계 및 구조 작성 (Django / Flask / FastAPI 중 하나 선택)
•API 설계 문서 작성 (예: REST API 구조, 주요 엔드포인트 설계)
•데이터베이스 스키마 설계 초안 작성 (가게 정보, 사용자 정보, 인증 기록 등)

- 준비해올 것:
  •간단한 CRUD API 설계 샘플 (예: 가게 등록, 가게 조회, 인증 기록 저장)
  •데이터베이스 모델 구조 초안

(김시연) NLP 구현 공부해오기

⇒ 구현 난이도 어려움으로 포기

(이설후) 지도 api 구현 공부해오기

⇒ 진행

(서희정) OCR 공부해오기

⇒ tesseract OCR + microsoft Azure ocr 사용하기

- 해야할 것
  - 포인트와 기부 까지의 프로세스 구체화
  - 수익모델 어떻게 할것인지
  - 선행사례 2~3개, 네이버 마이플레이스, 외국에 있는지
  - 영수증 검증 방법 고민
  -

4/9 회의록

태그	팀회의
날짜	@2025년 4월 9일

회의 안건

- 이용자의 악용을 막고자 영수증 검증을 하기로 함

1. 영수증 검증 방식을 “자연어 처리”로 하기로 했으나 너무 어려워져 좀 더 간단한 방식을 사용하자는 의견 제기
    a. 사용자에게 INPUT으로 상호명을 입력하게 하고
    b. 24시간 내 영수증만 인정 / 3일에 1번씩 리뷰 작성 시간을 제한
    c. 영수증에 출력된 사업자 번호와 사업자 번호 조회 API 간의 정보 대조를 통해 확인 → 어려워지니 하지 말자
        i. 이거 ‘영수증에서 상호명을 어떻게 얻을지’랑 ‘도용 방지 어떻게 할지’ 서로 다른 문제인데 구분해야 할 것 같습니다. NLP냐 사업자번호냐 input이냐는 전자, 24시간 내 영수증만 사용가능하게 + 한 사용자가 한 가게에 3개월에 한 번씩만 쓸 수 있게는 후자였어요. 근데 생각해보니까 input이나 nlp로 하면 상호명이 같은 가게도 있을 수 있겠네요. 사업자번호는 영수증에 사업자번호가 없는 경우를 어떻게 해야할지 생각해봐야 할 것 같습니다.
2. 의견 차이 : 실제
    a. 먼저 완성을 위해 소규모의 데이터와 최소한의 구현 기능으로 프로젝트의 목표 달성에 집중하자는 의견
    b. 소규모의 데이터로 하더라도, API나 다른 기능은 기존 프로세스대로 구현하자는 의견

▼ 이설후

- 영수증 검증방식
    - 이미지유사도 필터링 방식 + 시간 기반 인증제도
    - 전체 인증 흐름

[1] 사용자가 영수증 이미지 업로드
↓
[2] OCR 수행
↓
[3] 결제일시와 업로드 시간 비교 → 24시간 이내인지 확인
↓
[4] 이미지 유사도 검사 → 기존 DB에 있는 이미지와 비교
↓
[5] 조건 통과 시 인증 완료 → 포인트 지급

▼ 서희정

1. 기부 프로세스
    a. 기부 목록 리스트 : 기부 단체명과 기부하기 버튼을 누르면 포인트 기부하기가 뜨도록 하기 → 펀딩처럼 일정 포인트양이 채워진다면 “목표 달성!” → 추후 실제로 기부 증서와 기부자들에게 뱃지나 매달(자랑할 수 있는 형식으로 만들기)
    b. 포켓몬처럼 스타팅 포켓몬 만들기 (기부의 게임화) → 0~2500포인트 : 유아, 2500~5000포인트 : 청소년, 5000~ 10000 (구간이 왜이래) : 성체 → 성체가 된다면 그동안 감사했습니다라는 문구와 함께 포인트 전환 → 다음 2세대 포켓몬 소개
    c. 길드 랭킹 : 학교(특히 사회시간에 자주 쓰일 수 있을 듯), 회사, 단체 등등 길드를 만들어 기부 랭킹을 만들기.

2. 우리 서비스의 지속적인 매력포인트
    a. sns 공유 기능 (특히 인스타그램과 쓰레드, 당근(느낌)을 중심으로)
    b. 리뷰 공개 및 (지역 기반) 커뮤니티 만들기 (배달의 민족에 사장님 리뷰를 댓글처럼) , 공감기
    c. 스토리형식, 피크민 게임처럼 평상시에도 키우는 맛이 나게 레벨업 형식 가미
    d. 특정 테마 시즌에 맞춰 추천하기 ： 착한 맛집들을 단순 나열하는 것이 아닌 계절, 맛집, 특히 더 급한 가게 등등

3. 선한 가게의 기준 : 기부를 많이? 어느 정도 해야 선행 가게 리스트에 올라갈 수 있는가? →그럼 선행순위는 어떻게 매기는가?

4. 돈쭐낸다의 중의적 의미 : 가격이 착하고 사장님이 망할 것 같으면 돈쭐내자고도 하지 않는가 같이 연계 및 구분해서 ?
5. 리뷰 말고 추가적으로 포인트 얻는 법
6. 영수증 검증 :
   영수증 업로드 시
   자르기(Crop) 기능 제공 → "가게 정보 영역"만 선택하게 하기

   추출 실패 시 어떻게 할 것인지 → 다시시도하라는 메세지 ? 아니면 수동으로 입력하게 할 것인지

폰트 크기 기반 인식 (Google Vision / Kakao OCR 가능)

- 상호명은 대개 **폰트가 큼** → OCR 결과에서 **bounding box 크기가 큰 텍스트 우선 추출**
- Kakao OCR API는 bounding box 정보도 함께 제공하므로,
  - **텍스트 + 위치 정보**를 조합하여
  - "상단 20% 안에 위치 + 가장 큰 글씨" → 상호명 후보로 간주 가능

착한 가게 리스트와 유사도 비교 (Fuzzy Matching)

- OCR 결과에서 **후보 단어들을 리스트로 추출**한 뒤,
- **착한 가게 DB 상호명 리스트와 유사도 매칭** (Levenshtein 거리 등 활용)

GPS 참조

(강병진) 문서 탭에 핵심기능 API 명세서 초안 작성했습니다.

선행기술 + 사례 조사 매우 중요 !!

제안발표에서 최소한 유스케이스는 그리자

제약조건: 사용자 타겟 구체화, 지원환경도, 개발환경,,,

추진일정: 간트 차트 만들자

PPT: 제안서 양식이랑 비슷하게 만드는 걸 선호하시는 듯

교수님 발표 피드백

→ 월요일에는 논의가 제대로 되어야할 것 같다./ 오늘도 해서 완성도를 높이자.

최초 제안 시: 영수증 ocr 텍스트 추출 + 등록하는 상호명과 대조 → 내부 DB에 영수증 양식 저장

이후 리뷰 시: 기존 영수증 양식과 대조

분석 + 시각화 / 기획도 잘하고 기능도 잘하고

데이터랑 통계 문제 이슈를 ,,

0. 가게 리스트(배너)
1. sns 공유 기능 - 그림 생성
2. 지도 기능
3. 리뷰 쓰기, 커뮤니티 기능
4. 모바일 웹페이지 기능
5. 기부 프로세스 - 캐릭터 양육 느낌으로
6. 2배 기능
7. 마이페이지

피그마 디자인 1명 (프로토타입 제작) : 토요일 자정까지
수행 계획서 작성 2명 : 토요일 2400까지 -
PPT 발표 자료 제작 1명 : 일~ 화요일 저녁 8시까지 -

4/14 회의록

태그	팀회의
날짜	@2025년 4월 14일

▼ 회의 준비

[미리 읽어야 할 자료나 결정해야 할 사항을 적어주세요]

회의 안건

1. 수행 계획서 피드백 : 교수님 양식 목표와 비교하며 다시 읽어보기
    a. 개발 내용 구체화 (좀더 자세하게)
    b. 구현 계획 다같이 읽어본다음 수정
    c. 설계의 제한요소 다시 작성하기
    d. 플로차트 작성하기
    e. 2.5 세부 작업에 대한 간트차트 확인 및 세부 작업 별 구성원 역할 정하기
2. 디자인 피드백
3. 피피티 초안

커뮤니티 제보 투표기능

투표는 따로 추가적인 페이지 없이 커뮤니티 내 가게 제보 리뷰에서 좋아요/싫어요 기능으로 투표 대체

기준 : 좋아요의 개수가 100개 넘고, 싫어요 비율이 절반 이하인 경우의 게시물들만 통과 >> 1차 투표

기준 달성 시 바로 돈쭐 가게 후보 리스트에 올라가게 하자 vs 한 달이나 일주일이라는 정량적인 시간을 두고 올라가게 하자

⇒ 정량 시간을 두기로 함. 커뮤니티 특성 상 여론이 반영되기에 시간이 필요하다는 이유

가게 리뷰 탭 선정

1. 가게 리뷰를 커뮤니티 탭에서 보이게 할 것이냐

vs 착한 가게 탭에서 보이게 할 것이냐

2. 가게 평점을 별점으로 할 것이냐 추천/비추천으로 할 것이냐

⇒ 결정 못 함

리뷰 별로 추천 비추천이 달림 → 그럼 해당 리뷰가 부적절하다고 생각해 비추천이 달릴 수도 있음

단순 추천/비추천으로 가게 평점을 정하기엔 명확하지 않음

ex. 평범한 가게인 경우 별 3점 주기 → 추천/비추천의 경우 : 어떤 걸 선택해야하는가?

플로우 차트 제외. 비즈니스적 측면을 심층 분석할 것

회의 결과

1. 가게 리뷰를 착한 가게 탭에서 보이게 하기
2. 좋아요/싫어요 기능으로 제보 투표 대체
3. 선행 가게 모니터링 , 별점 논의는 추후 좀더 디밸롭하기

다음 회의

일정: 2025년 04월 16일 16:00

준비:

병진 : 시스템구조도 작성, 수행 계획서 수정

시연 : 선행연구표, 시퀀스 다이어그램 작성

설후 : 발표자료 완성

희정 : 회의록, 깃허브 수정, 전반적인 피드백